

# Premise-Bound Execution

## Approval Validity under Mutable World State in an Agent Runtime\*

MØBIUS

Draft — September 15, 2026

### Abstract

An agent runtime that changes the world must decide, before each effect, whether the justification still holds. We measure it across 272 arms in a runtime composing two providers over one filesystem root, spanning one run, a run boundary, and a resume. A recorded human approval authorises an effect after its justifying world has changed — across a reload of its file-backed journal, and where the runtime looks, sees the change, and proceeds. **Every gate on that path returns a locally correct answer.** On the shipped daemon, through a real client, an approval also authorised a re-proposal whose content the person was never shown. There, too, the repair refused a stale approval over a top-level file and executed one over a nested file its look did not name. Freshness does not explain it: a provider-local revision is authoritative only for its own history, so a second writer moves no counter and the guard returns on zero drift before its stricter mode; the enforcement repair already chosen for this is inert as specified, its resolver invoked zero times. Durability does not explain it either: a control shows an observation-derived premise — session, revision, evidence — ceasing to rule once its evidence is gone, where an approval records capability, resources and a timestamp nothing compares. What separates them is whether a record carries enough to decide that its justification can still rule. Measurement eliminates three of four candidate anchors, leaving resource identity recorded on the approval. Building it exposed representation defects its design review had not: authority and evidence are not independent planes; one resource list cannot serve both permission and binding; a single-use rule was implemented as zero-use; and a look that said nothing about a path was read as the path being absent — an error a green suite and a passing battery both endorsed.

## 1 Introduction

An agent runtime asks a person to approve an effect, records the answer, and later reaches a second effect the recorded answer covers. Between the two the world can change, and here a second writer changes it. So:

*When an old approval is about to authorise a new effect, what fact does the runtime check to establish that the world which justified that approval still holds?*

Measured on this runtime, **none of the gates on that path asks that question** (Figure 1). The precondition gate asks whether what was observed was observed at the session’s current revision. The authority gate asks whether a recorded decision names this capability and these resources. The staleness guard asks whether the provider’s revision has moved. Each returns a true answer; the effect is dispatched; the approved world is gone.

---

\*Subtitle provisional. The anchor surviving §6’s elimination has since been implemented, and §7 reports what building it revealed.



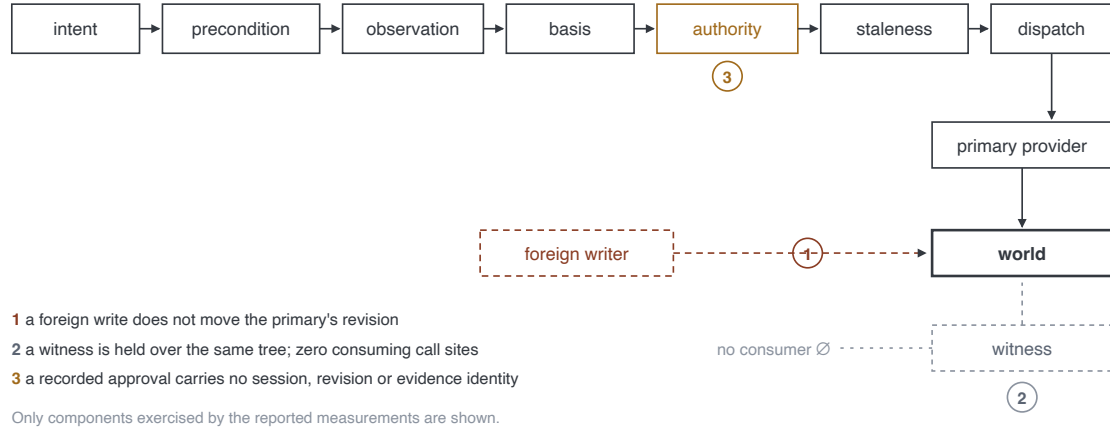
**Figure 1.** A human approval recorded against  $W_0$  authorises a write dispatched against  $W_1$ . The runtime observed  $W_1$  and recorded its content; the precondition gate, the authority gate and the staleness guard each return a true answer, and none asks whether the approved world still holds. The provider-local revision is unchanged throughout, because the mutation was made by a second writer. Measured on production runtime code driven by a test harness, with actions built by hand rather than compiled from an intent (§3).

That an agent may act on stale state is known and is not our claim; recent work names stale authorization as a failure mode of agentic systems and gives a correctness condition for it. Our claim is about *why a system that checks can still miss*, and it has two halves that the paper keeps apart. The freshness mechanism is correct, is run, and returns a *true* answer that is nonetheless the wrong answer, because **the observer it consults cannot express the change**: the guard reports that the world is still at revision  $N$ , and it is — the file was rewritten by someone else. And the authority record is not wrong at all, because **it carries no binding to the evidence that justified it**, so there is nothing in it for a later run to re-check. The anomaly is therefore not *the runtime forgot to check freshness*. It is that the runtime checked freshness with a representation that could not express the relevant change, while authority carried no binding to the evidence that justified it.

**Why this setting is worth measuring rather than arguing.** The runtime already holds an observer that could answer: a second provider is composed over the same working tree and kept open for the run, and the architecture record introducing such witnesses says they exist to be asked — yet the field holding it has zero consuming call sites in production code. The repair the system had already chosen for this does nothing: implemented in full, it changes no behaviour. And one of the paths has a person on it, where under the contract the runtime states, every measured behaviour is correct.

**Contributions, as one chain.** The paper is a single line of measurement rather than a set of findings, and we state it in that order.

1. **The anomaly.** A recorded approval authorises an effect after its justifying world has changed, including across a file-backed resume and including the arm where the runtime looks, sees the change, and proceeds (§4).
2. **Two causes, not one.** The same permissive verdict is reached by two routes that share no input: one builds no premise to compare, and the other builds one whose comparison cannot express a foreign write. Observing verdicts alone cannot tell them apart (§4, §5).



**Figure 2.** The execution path from intent to effect, with the three seams where measurements landed: (1) a mutation by a second writer does not move the primary's revision; (2) a witness is held over the same tree with no consumer in production code; (3) a recorded approval carries no session, revision or evidence identity. Components not exercised by these measurements are omitted. The runtime shown is the one the anomaly was measured on: seam (3) is what §7 closes, and an approval recorded by the repaired runtime carries the identity of each resource it was granted over.

3. **A control that rejects the general reading.** The blindness is not a property of durable decisions. An observation-derived premise records session, revision and evidence and reports `unevaluable` when it can no longer rule; an approval, *as the runtime recorded one when this was measured*, records nothing that could be gone (§4). §7 is what happens when that missing anchor is added.
4. **Elimination by measurement.** Three of four candidate validity anchors are ruled out by measurement rather than preference, leaving resource identity recorded on the approval (§6).
5. **What implementing the survivor exposed.** Building it contradicted four things its own design review had asserted, each at the level of representation rather than of code (§7).

## 2 System model

Definitions are given only for the concepts the measurements use: *provider*, *session*, *world*, *witness*, *observation*, *revision*, *basis*, *premise*, *authority* and `UserDecision`, *dispatch*, and *persistence/resume*.

A *revision* is a per-session counter recording what *that provider did*: reads advance a separate observation counter, and it moves only when the provider commits an effect of its own. This definition is stated in the architecture record, not inferred here, and the paper follows from it.

*Premise* is used in its ordinary logical sense — a fact about the world that must still hold for an effect to be permitted. This is not the sense used in machine-assisted theorem proving, where a premise is an existing lemma retrieved from a library.

## 3 Experimental setup

Arms marked *daemon* run the shipped binary, answered by a real client. Every other arm runs production runtime code under a test harness (the real loop, coordinator and journal, with scripted

intents), and arms marked gate-level hand the gate an action built by hand. Substrate: two providers over one filesystem root, one machine, a single writer, scripted intents. Commits are named per experiment. Each experiment registers numbered predictions before measuring and retains refuted ones; each ships a mutation battery, and a surviving mutation is answered as one of *untested branch*, *weak assertion*, or *behaviourally identical*.

**The substrate, stated honestly.** At the head of this line (9b225b6), the suite is 227 files and 1811 passing tests, with one skipped and none failing. That run was made by the paper track rather than the experiment track, on a clean tree, and it includes the check of Appendix B that every arm's recorded commit touched its experiment.

**It was not green two slices ago, and the correction to why is the part worth recording.** One pin compares the project's own ownership ledger — a document naming which slices have written a jointly-held package — against the git history of that package. Two slices wrote the package in one commit and added the ledger row in the next, and the two tracks measured on opposite sides of that gap: one reported green from before the code commit, the other red from after.

Our first account of this called the gap structural, on the analogy of a second pin whose gap genuinely is: that one records a commit hash, and a hash cannot be written into the commit it names. **The analogy is false and the experiment track rejected it.** Nothing stops the ledger row from travelling in the same commit as the code, and the slice after these two did exactly that. The gap was a habit, and calling it a design property would have made a practice look like a constraint. **This is the third time in this study that two things were merged because they had the same shape** (§5, Appendix A). The description was not invented here and was correct where it was first made — of the hash-recording pin, whose gap is real. It was then carried to a second pin without being re-derived there, and **that carry is the merge**: a true sentence about one object, reused for an adjacent one, is a summary of the kind this study reports being misled by elsewhere, differing only in that this summary is a correct sentence about something else.

The measurement lesson is the converse of one recorded earlier. That one said not to run a suite against another agent's uncommitted tree; this one says a green taken *before* a commit is a green taken at a moment that could not have shown the defect, so the run must be repeated after it. **A study that only ever quotes a green suite is quoting a chosen moment** — and here the moment was chosen by us rather than found. The suite is green when uncontended. Under load a class of tests that start real processes or sockets can be starved at the default five-second timeout, and which of them lose is nondeterministic — consecutive runs gave zero and three failures, the three being different files. We do not raise the timeout: a timeout increased to quiet a suite is how a real slowdown is hidden. Separately, an earlier standing count of failures reported alongside these experiments was an artifact of an execution sandbox rather than a property of the tree; no measurement in this paper depends on it, because the mutation-battery runner refuses a red baseline by construction.

**Two compile censuses.** The first re-checked every compile through the harness (163 compiles) with the shipped compiler's schema check, after first confirming that the check refuses a known-bad input. The second, pre-registered before its hook existed, checked every action in all thirty-one experiments, whether compiled or built by hand. **The anomaly (V59) and the journal reload (V61) compile through the shipped compiler**, with enforcement on. The first census had labelled them hand-built; it is kept, with its coverage table corrected in four places. The matching repair's arms (V68) are accepted. The matching failure's command arms (V67) are accepted by the shell provider's own schema, but shell is not composed into the daemon, so they are unjudged for the shipped path. Refused inputs are confined to three experiments: the cost measurement (all its arms), one patch arm, and the repair's gate-level comparison, where every write passes a path

without its content. **Fifteen of the twenty-two cells behind the repair’s headline are therefore counterfactual.** For nine, outcome, rule and route are measured unchanged once content is added **in both columns** — the baseline was re-run under each repair and reproduces the ledger’s column 26/26 — so those cells’ movement between columns is measured on an input the checker accepts; six rest on inputs no compile produces. The seven others, the five loop arms and two commits, compile as shipped. One arm has no record and is reported as not censused.

**What the numbers in this paper are.** Counts here are of five kinds, and we name the kind wherever one could be read as another. An *arm count* says how many experimental conditions were run; it is a property of experimental design. A *control count* and a *mutation count* are the same kind of fact about the instruments. An *observation-event count* is something a real execution produced — facts returned by a look, call sites a dispatch reached, receipts a capability emitted. A *derived figure* is computed from a rule rather than read off a measurement, and is labelled as derived at the point of use.

**No number in this paper is a workload statistic.** We ran no workload, so nothing here states how often anything happens in a running system. In particular the distribution of arms across the two causal branches describes what we chose to measure, not what a runtime does: *nine arms followed branch A* is a fact about this study and would not support *most approvals have no premise*.

## 4 Results

Each result opens with its status. OBSERVED is measured in one experiment; REPLICATED in more than one independent experiment, with the count stated; REFUTED is a prediction the measurement contradicted, retained rather than removed; and COUNTERFACTUAL evaluates a real component under a mode the runtime does not produce and is therefore not evidence of runtime behaviour. Nothing here is marked with a status it did not earn: most of these results were measured once, and are labelled that way.

Organised by research question. Each subsection follows: question, setup, control, measurement, result, interpretation, boundary. Figure 4 gives every measured arm.

### 4.1 RQ1 — When is a premise actually constructed?

REPLICATED An action carries a premise exactly when its proposer declares one. With no declared precondition the dispatched action carries *no basis at all* and the guard exits at “*names no basis*”: no revision comparison occurs. The declaration is the sole source of the basis’s targets; observations contribute none.

### 4.2 RQ2 — Can a provider-local revision detect a foreign mutation?

REPLICATED No. A write from outside the primary leaves its revision unmoved across two real observations. The guard returns `proceed` on zero drift *before* reaching the freshness mode switch, so `strict`, `revalidate` and `advisory` are indistinguishable here. Drift at a declared dispatch is structurally zero under this scheduler, which satisfies preconditions just-in-time.

### 4.3 RQ3 — Is the intended enforcement path reachable?

COUNTERFACTUAL Unrelated self-caused drift does open it, and a resolver reporting the target lost produces a refusal — but that input is injected by the harness and is not produced by this scheduler.

#### 4.4 RQ3b — Why the intended enforcement repair is unreachable

COUNTERFACTUAL With both halves of the decided repair implemented, the verdict is `proceed` and the resolver is invoked **zero** times; a control at forced drift invokes it once and refuses. The decision states two conditions it believes jointly sufficient; a third — that anything reaches the switch — is unstated, and could not have been stated, because the measurements establishing it came later.

#### 4.5 RQ4 — Can an approval outlive the world that justified it?

OBSERVED Yes (Figure 1). Across five arms, the unrelated-change arm and the experimental arm agree on every observable: the approval is not defeated by change at all. With a declared premise the runtime looks, records the rewritten content, satisfies its own premise with it, and dispatches on the old approval. Declaring a premise cannot help by specification: the gate asks whether the subject was observed *at the session’s current revision*, not whether what was observed still supports the action.

#### 4.6 RQ5 — Is the blindness general to durable decisions?

REFUTED The generalisation does not hold (Figure 3). An observation-derived premise carried across a run boundary returns `unevaluable` and withholds when its session is gone; a control distinguishes this from `unsatisfied`. Authority is the outlier on this axis, not the representative case. *Boundary: a run boundary, not a persist/resume cycle.*

**This control is what keeps the paper’s claim from being a much weaker one.** Without it the result reads *durable decisions go stale*, which is both known and not what was measured. What was measured is that two durable records in the same runtime, carried across the same boundary, behave differently, and the difference is legible in what each one holds. **The middle column describes the approval as this runtime recorded one at the time of the measurement**; the repair of §7 adds to it exactly the anchor whose absence the column reports, which is why the column is stated in the past rather than as a property of approvals:

|                           | observation-derived<br>premise                        | recorded approval                        | evidence bound to<br>a criterion      |
|---------------------------|-------------------------------------------------------|------------------------------------------|---------------------------------------|
| recorded                  | session, revision,<br>evidence                        | capability, resources,<br>decided-at     | session, revision,<br>observation id  |
| can report                | satisfied / unsatisfied<br>/ <code>unevaluable</code> | matched / not matched                    | —                                     |
| when its world<br>is gone | withholds, and<br>says why                            | cannot detect that the<br>question arose | rebinds, and the run<br>does not look |

**The third column is why we no longer state this as a property of the record alone.** An earlier version of this paper said the relevant property was *whether a record contains enough to determine whether its own justification can still rule* — which separates the first two cleanly: the premise’s session and revision suffice, since their disappearance is itself the answer, while none of the approval’s three fields can become false, so a timestamp nobody compares against a clock buys nothing.

The third record defeats that formulation. It carries the *same* provenance the first one revalidates with — the session it was captured in, the world revision current at capture, and the identifier of the observation that produced it. That is asserted of the record the measured run actually bound, not inferred from the type: the arm reads the evidence out of the resolved unknown and requires all three to be present. On the path that rebinds it to a criterion in a later run, **none of the three is**

**read.** The condition for rebinding is only that the question was once answered. So the corrected statement is in two parts: a record must carry enough to determine whether its justification still rules, **and something must ask.** The first is necessary and is not sufficient, and the three cases here are one of each — carries and is asked, does not carry, carries and is not asked.

**A third condition, measured after the other two.** Carrying provenance and being asked are still not sufficient, because **what is recorded has to be comparable.** The premise's basis is built as a resource paired with an identity. The evidence rebound to a criterion, on the path this study measured, names a resource and carries *no* identity — and not by accident of payload: a fact is minted from a *path*, and an identity only when a *digest* accompanies it, and the look that answers a criterion here returns hits with a path and no digest. A control takes a different look from the *same provider* and does obtain identities, so the cause is the shape of the look rather than the provider, the helper, or the fixture.

The consequence is a constraint on any repair, and we state it as a constraint rather than a recommendation. **On this path a later observation can replace a justification and can never confirm one,** because there is nothing recorded for it to agree with. And the two obvious ways to obtain something comparable trade off against each other: repeating the look that answers the criterion yields another incomparable hit, while taking the look that yields an identity answers a different question — *what is this file's digest* in place of *what does its content say*.

**We first reported that split as holding between capabilities, and a later measurement made it finer.** It runs between *branches*, some of them inside one function. The path that reads a file has three: a directory listing carries a digest in its structured state and mints no evidence at all; an inline read mints evidence without one; and a read whose content is withheld — binary, or past an inline size limit — mints evidence carrying path, digest and length together, which is comparable. **So both modes are reachable, and a recovery mechanism has to distinguish a justification it confirmed from one it replaced rather than assuming it can only ever do the second.** What ought to happen next is a decision about the product and was taken as one; it is not something these measurements settle. *Scope: one provider, two capabilities, three branches. How often a real goal takes one branch rather than another is a frequency question and is not measured here.*

**And the check that would ask is already written.** At the boundary where the third record fails — a resume, where the session that produced the evidence is gone — the first record's gate reaches *unevaluatable* from an identifier lookup, *before* the revision comparison it also performs: the originating session is absent from the session list, and the gate declines to assume that a counter it can no longer see never moved. That is the whole of it at this boundary. No look, no observation budget, no field that is not already recorded — and the identifier it looks up is one of the three the third record carries. The state it reads comes from the coordinator rather than the cognitive projection the seed iterates, so it is not the same call; but the coordinator is in hand at that point in the loop and is already queried by session identifier a few hundred lines away.

**So what separates the first record from the third is not representation, feasibility, or cost.** They carry the same fields, the check is the same check, and it is implemented in the same runtime for the other one.

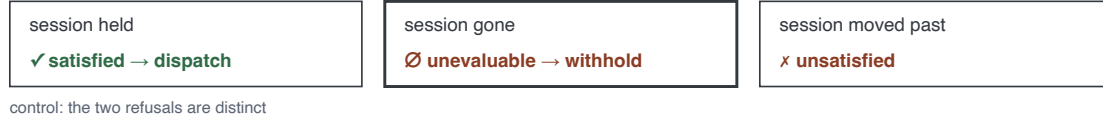
**It does not follow that connecting them would be enough,** and the reason is the conflation this paper is about, appearing once more. The verdict would land on the plane deciding whether a criterion is met, and that plane returns *two* values — satisfied or not, with a prose gap. The third value is not missing from the system. It is **delivered to that plane and folded away there:** the assertion evaluator returns *undecidable* distinctly from *violated*, and the criterion evaluator folds both into *satisfied: false*, preserving the difference only in prose that its consumer does not branch on.

**Two things make this more than an omission.** First, the file computing that distinction states

(a) WHAT EACH DECISION RECORDS



(b) OBSERVATION-DERIVED PREMISE, THREE ARMS



(c) USERDECISION, SAME THREE CONDITIONS



ONE CAN STOP RULING. THE OTHER CANNOT NOTICE THE QUESTION AROSE.

**Figure 3.** Two decisions the runtime carries across time, and what each records. The one anchored to a session and a revision can report that it is no longer able to rule; the one anchored to a capability and a resource set cannot detect that the question arose.

preventing exactly this fold as its reason for existing — *an assertion that cannot be evaluated is undecidable, not failed*, because reporting absence of evidence as evidence of absence is how a runtime manufactures verdicts it did not earn — and its own fold keeps `undetermined` as a peer of `failed` rather than collapsing it. Second, that verdict is *first-class elsewhere in the same runtime*. Counting only executable code, and excluding the files that merely name it in a comment or a message string, **seven modules across three packages act on the distinction**: the comparator produces it, a planner assigns it and uses it to decide whether a fresh observation is worth paying for, a classifier switches on it, a diagnosis filter selects it, an experience model and an effect ledger tally it, and a calibration routine removes it from its denominator *by construction rather than by a caller remembering to subtract it*. The eighth module that touches it is the criterion evaluator, which folds it away — **in the same package as the file whose opening lines forbid exactly that**. The value is computed, documented as load-bearing, and acted on everywhere except the one boundary the third record would have to report into.

So the repair is *smaller* than introducing a third value and the charge against the current state is heavier: nothing needs inventing, and something already delivered is being discarded. This paragraph is read from the source rather than measured, and we mark it as such; it is a further instance of an absence stored as a value, kept out of the count in §5 for that reason.

We report the distance as measured and do not propose closing it: what a criterion’s evidence turning `unevaluable` should *mean* is a separate question and a harder one than for a premise — a premise withholds one dispatch and looks again, whereas a criterion that cannot be evaluated bears on whether the goal can be completed at all.

#### 4.7 RQ6 — Does the result survive persistence and a journal reload?

**OBSERVED** Yes, on the real persistence path (Figure 5): a file-backed event log, a journal reopened per pass, and a second pass asserted to have reloaded from disk. In one arm the runtime shows both behaviours at once — the resumed run *could not* use the persisted observation and looked again, while the approval *was not* revalidated. Same journal, same reload, same session boundary, opposite treatment. A same-world control confirms that resume alone does not invalidate an approval,

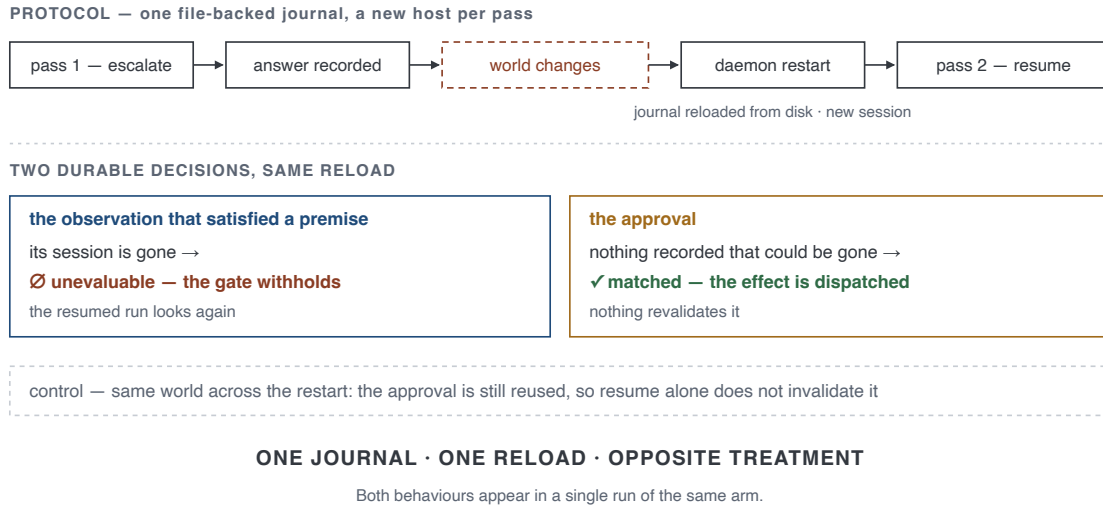


## ALL 272 MEASURED ARMS

MEASURED ✓ yes ✗ no ∅ the gate was not reached · not measured here — a missing measurement is never a measured false.

**WRONG** a prediction existed before the measurement and the measurement contradicted it; **SOURCE** names whose belief failed — a written *preregistration*, a written *design claim*, or an *unwritten* assumption held only by the code. **NONE** no prediction existed at all.**CTRL** control arm. **CF** counterfactual: a real component under a mode the runtime does not produce. **CAUSE** the demonstrated route to the verdict, where one was established.

| EXP                  | ARM                                                  | CAUSE | ROLE    | PREDICTION | SOURCE    | BASIS<br>BUILT | REVISION<br>MOVES | REVALIDATION | APPROVAL<br>REUSED | DISPATCH |
|----------------------|------------------------------------------------------|-------|---------|------------|-----------|----------------|-------------------|--------------|--------------------|----------|
| WITHIN RUN 122 arms  |                                                      |       |         |            |           |                |                   |              |                    |          |
| probe-3              | no-declaration                                       | A     | CTRL    |            |           | ✗              | -                 | ∅            | -                  | ✓        |
| v58                  | runtime-path/no-declaration                          | A     |         | WRONG      | prereg    | ✓              | -                 | ∅            | -                  | ✓        |
| probe-3              | declared-premise                                     | B     |         |            |           | ✓              | -                 | ✗            | -                  | ✓        |
| probe-4              | declared-premise/foreign-write                       | B     |         |            |           | ✓              | ✗                 | ✗            | -                  | ✓        |
| probe-5              | self-effect-then-foreign/advisory                    | B     |         |            |           | ✓              | ✓                 | ✗            | -                  | ✓        |
| probe-5              | no-self-effect/advisory                              | B     | CTRL    |            |           | ✓              | ✗                 | ✗            | -                  | -        |
| probe-5              | self-effect/revalidate-no-resolver                   | B     | CF      |            |           | ✓              | -                 | ✓            | -                  | -        |
| probe-5              | self-effect/revalidate-resolver-reports-lost         | B     | CF      |            |           | ✓              | -                 | ✗            | -                  | -        |
| probe-5              | self-effect/revalidate-resolver-reports-intact       | B     | CTRL CF |            |           | ✓              | -                 | ✗            | -                  | -        |
| probe-6              | single-declared-intent                               | B     |         |            |           | ✓              | ✗                 | ✗            | -                  | -        |
| probe-6              | undecleared-then-declared/scheduler-order            | B     |         |            |           | ✓              | ✗                 | -            | -                  | -        |
| probe-6              | declared-then-undecleared/scheduler-order            | B     | CTRL    |            |           | ✓              | ✗                 | -            | -                  | -        |
| probe-6              | runtime-basis/revalidate-with-resolver               | B     | CF      |            |           | ✓              | ✗                 | ✗            | -                  | -        |
| probe-6              | runtime-basis/revalidate-drift-forced                | B     | CTRL CF |            |           | ✓              | -                 | ✗            | -                  | -        |
| v58                  | fixture-strict-basis/fs.write                        | B     |         |            |           | ✓              | -                 | ✓            | -                  | ✓        |
| v58                  | fixture-strict-basis/fs.patch                        | B     | CTRL    |            |           | ✓              | -                 | -            | -                  | ✗        |
| e2e1                 | W2/CTRL                                              | CTRL  |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| e2e1                 | W2/2-REL                                             | CTRL  |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| e2e1                 | W2/2-IRR                                             |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| e2e1                 | W4/OUTSTANDING                                       |       |         |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/NARROW/N=1                                     |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/NARROW/N=2                                     |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/NARROW/N=3                                     |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/NARROW/N=5                                     |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/NARROW/N=8                                     |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/WIDE/N=1                                       |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/WIDE/N=2                                       |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/WIDE/N=3                                       |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/WIDE/N=5                                       |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| ov1                  | FIRST/WIDE/N=8                                       |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| probe-4              | witness-queried-at-dispatch/foreign-write            |       |         |            |           | -              | -                 | -            | -                  | -        |
| probe-4              | witness-queried-at-dispatch/no-foreign-write         |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| probe-4              | foreign-write/counter                                |       |         |            |           | -              | ✗                 | -            | -                  | -        |
| probe-4              | own-provider-writes/counter                          |       | CTRL    |            |           | -              | ✓                 | -            | -                  | -        |
| u6                   | C2/control/never-asked-twice                         |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v62                  | detector/no-dependent-task                           |       |         |            |           | -              | -                 | -            | -                  | -        |
| v62                  | detector/task-depends-on-it                          |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v62                  | detector/task-depends-on-other                       |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v63                  | identity-absent/vs-unmoved                           |       |         |            |           | -              | -                 | -            | -                  | -        |
| v63                  | identity-known-and-moved                             |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v63                  | identity-absent/no-baseline                          |       |         |            |           | -              | -                 | -            | -                  | -        |
| v63                  | fs.observe/file-read                                 |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v63                  | fs.observe/directory-listing                         |       |         |            |           | -              | -                 | -            | -                  | -        |
| v64                  | receipt/fs.write                                     |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v64                  | receipt/fs.mkdir                                     |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| v64                  | receipt/fs.delete                                    |       |         |            |           | -              | -                 | -            | -                  | -        |
| v65                  | legacy/no-bindings/within-run                        |       | CTRL    |            |           | -              | -                 | -            | ✓                  | ✓        |
| v65                  | scope/prose-target-in-resources                      |       |         | WRONG      | design    | -              | -                 | -            | ✓                  | ✗        |
| v65                  | uncovered-look/nested-path                           |       |         | WRONG      | unwritten | -              | -                 | -            | ✓                  | ✗        |
| v66                  | A/fs.write/path-exists                               |       |         |            |           | -              | -                 | -            | -                  | -        |
| v66                  | B/fs.write/path-absent                               |       |         |            |           | -              | -                 | -            | -                  | -        |
| v66                  | C/fs.patch                                           |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v66                  | D/fs.delete/file                                     |       |         |            |           | -              | -                 | -            | -                  | -        |
| v66                  | E/fs.mkdir                                           |       |         |            |           | -              | -                 | -            | -                  | -        |
| v66                  | F/fs.move/destination-absent                         |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| v66                  | G/fs.delete/directory                                |       |         |            |           | -              | -                 | -            | -                  | -        |
| v66                  | H/fs.move/destination-exists                         |       | CTRL    | NONE       |           | -              | -                 | -            | -                  | -        |
| v66                  | I/fs.move/broad-look                                 |       | CTRL    | NONE       |           | -              | -                 | -            | -                  | -        |
| v67                  | census/repo.stage                                    |       |         |            |           | -              | -                 | -            | -                  | -        |
| v67                  | census/repo.commit.create                            |       |         |            |           | -              | -                 | -            | -                  | -        |
| v67                  | census/repo.push                                     |       |         |            |           | -              | -                 | -            | -                  | -        |
| v67                  | census/shell.run                                     |       |         |            |           | -              | -                 | -            | -                  | -        |
| v67                  | census/shell.process.start                           |       | CTRL    | NONE       |           | -              | -                 | -            | -                  | -        |
| v67                  | empty-resources/shell.run/other-command              |       |         | NONE       |           | -              | -                 | -            | ✓                  | -        |
| v67                  | empty-resources/repo.commit.create/other-commit      |       |         | NONE       |           | -              | -                 | -            | ✓                  | -        |
| v67                  | empty-resources/rescued-by-differing-target          |       | CTRL    | NONE       |           | -              | -                 | -            | ✗                  | -        |
| v67                  | empty-resources/shared-target                        |       | CTRL    | NONE       |           | -              | -                 | -            | ✓                  | -        |
| v68                  | empty-resources/other-command                        |       |         |            |           | -              | -                 | -            | ✗                  | -        |
| v68                  | empty-resources/same-command                         |       | CTRL    |            |           | -              | -                 | -            | ✓                  | -        |
| v68                  | empty-resources/other-commit                         |       |         |            |           | -              | -                 | -            | ✗                  | -        |
| v68                  | named-resource/unchanged                             |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v68                  | shared-prose-target/two-commands                     |       |         | WRONG      | prereg    | -              | -                 | -            | ✓                  | -        |
| v68                  | legacy-decision/no-occurrence                        |       |         | NONE       |           | -              | -                 | -            | ✗                  | -        |
| v68                  | question-id/occurrence-ignored-when-resources-nam... |       | CTRL    | NONE       |           | -              | -                 | -            | -                  | -        |
| v69                  | coversTargets/production-consumers                   |       |         |            |           | -              | -                 | -            | -                  | -        |
| v69                  | covering/fs.move/destination-exists                  |       |         |            |           | -              | -                 | -            | -                  | -        |
| v69                  | covering/fs.move/destination-absent                  |       |         |            |           | -              | -                 | -            | -                  | -        |
| v69                  | covering/single-resource                             |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v69                  | covering/no-progress                                 |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v69                  | revalidation/RV-ABSENT/targeting-honoured            |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| v69                  | revalidation/RV-ABSENT/targeting-ignored             |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v70                  | S1/named-then-enont                                  |       |         |            |           | -              | -                 | -            | -                  | -        |
| v70                  | S2/sweep-covers-unamed                               |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v70                  | S3/substring-mention                                 |       |         |            |           | -              | -                 | -            | -                  | -        |
| v70                  | S4/flag-reads-the-request                            |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-1                | S1/seeded-evidence/comparability                     |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-1                | S2/fs.observe-listing                                |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v72-1                | S3/which-key-each-half-records                       |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-1                | S4/no-path-yields-no-fact                            |       | CTRL    | NONE       |           | -              | -                 | -            | -                  | -        |
| v72-2                | S4/held-assertion                                    |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v72-2                | S1/undecidable-assertion                             |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-2                | S2/violated-assertion                                |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-2                | S3/field-for-field                                   |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-3                | S1/listing/evidence                                  |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| v72-3                | S3/search/evidence                                   |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v72-3                | S4/spilled-read/evidence                             |       |         | NONE       |           | -              | -                 | -            | -                  | -        |
| v72-4                | S3/user-judged/answered                              |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v72-4                | S1/declined-vs-unasked                               |       |         |            |           | -              | -                 | -            | -                  | -        |
| v72-4                | S4/declined-evidence-dropped                         |       |         | WRONG      | prereg    | -              | -                 | -            | -                  | -        |
| v73                  | K1/no-decision                                       |       | CTRL CF |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | A1/bound/unchanged                                   |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | A2/bound/changed                                     |       | CF      |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | A3/bound/identity-unobtainable                       |       | CF      |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | A4/absence-reason/first-use                          |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | A5/absence-reason/already-used                       |       | CF      |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | A6/legacy/within-run                                 |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | A1/no-addressable                                    |       |         |            |           | -              | -                 | -            | -                  | -        |
| v73                  | B1/names-nothing/other-occurrence                    |       |         |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | B2/names-nothing/same-occurrence                     |       |         |            |           | -              | -                 | -            | -                  | -        |
| v73                  | B3/names-a-resource/same                             |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| v73                  | B4/names-a-resource/different                        |       | CF      | WRONG      | prereg    | -              | -                 | -            | ✗                  | -        |
| v73                  | B5/legacy/no-occurrence                              |       |         |            |           | -              | -                 | -            | ✗                  | -        |
| v73                  | E1/no-second-look/first-use                          |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | E1/no-second-look/later-use                          |       | CF      | WRONG      | prereg    | -              | -                 | -            | ✗                  | -        |
| v73                  | E1/second-look/first-use                             |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | E1/second-look/later-use                             |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| v73                  | G1/prose-target-only                                 |       | CF      |            |           | -              | -                 | -            | -                  | -        |
| v73                  | C1/single-resource                                   |       | CTRL    |            |           | -              | -                 | -            | -                  | -        |
| v73                  | C2/two-resources/both-exist                          |       |         |            |           | -              | -                 | -            | -                  | -        |
| v73                  | C3/two-resources/destination-absent                  |       |         |            |           | -              | -                 | -            | -                  | -        |
| v73                  | C4/two-resources/port-ignores-targeting              |       | CTRL CF |            |           | -              | -                 | -            | -                  | -        |
| v73                  | D1/mentionsTargets/production-consumers              |       |         |            |           | -              | -                 | -            | -                  | -        |
| RUN BOUNDARY 13 arms |                                                      |       |         |            |           |                |                   |              |                    |          |
| v59-1                | approval-reuse/no-mutation                           | A     | CTRL    |            |           | ✗              | -                 | -            | ✓                  | ✓        |
| v59-1                | approval-reuse/foreign-write                         | A     |         |            |           | ✗              | -                 | ∅            | ✓                  | ✓        |
| v59-1                | approval-reuse/irrelevant-foreign-write              | A     | CTRL    |            |           | ✗              | -                 | -            | ✓                  | ✓        |
| v59-2                | declared-premise/foreign-write                       | B     |         |            |           | ✓              | ✗                 | ✗            | ✓                  | ✓        |
| v60                  | gate/session-held                                    | B     | CTRL    |            |           | ✓              | ✗                 | ✗            | ✓                  | ✓        |
| v60                  | gate/session-vanished                                | B     |         |            |           | ✓              | -                 | ✓            | -                  | ✗        |
| v60                  | gate/session-moved-past                              | B     | CTRL    |            |           | ✓              | ✓                 | ✓            | -                  | ✗        |



**Figure 5.** The persistence protocol and its two arms. One journal is written to disk, a new host is opened per pass, and the second pass is asserted to have reloaded. The observation that satisfied a premise cannot be reused — its session is gone, so the gate reports *unevaluable* and the resumed run looks again — while the approval, having recorded nothing that could be missing, matches and dispatches. A same-world control shows the approval is reused across the reload even when nothing changed, so resume alone does not invalidate it.

so a repair may not simply expire every approval at every reload.

**What this boundary is on the daemon path, established after the measurement.** The daemon keeps one journal per turn, and an answer starts a second run over that turn’s journal; the reload measured here is that second run. No decision reaches a later turn, and a pending approval is held in memory and ended by a daemon restart. Both facts were read from source and then measured on the daemon (E2E-1, W4); neither changes the arms above, only what *restart* may be taken to mean.

#### 4.8 RQ7 — Does a detector exist, and what happens to what it detects?

**OBSERVED** One exists, is reached, and its output is discarded. The runtime holds a working identity-based revalidator: it compares a new observation’s facts against the identity a task depended on and returns *restored*, *invalidated* or *disputed*. It is live production code on the path every observation takes.

Given the approved file’s identity moving underneath a recorded approval, **the runtime notices and throws the notice away**. Every observation passes through one function, which asks the invalidator what changed and *returns before revalidating* when the change is depended on by no *task* — and an approval is not a task. Two controls fix the cause to dependency rather than to detection: a task depending on the changed resource yields a non-empty dependent list and is revalidated; a task depending on a *different* resource has the same change reported and still discarded.

**Across a resume the detection does not happen at all**, for a second and unrelated reason: the invalidator is a per-loop field, so a resumed run begins with an empty one and holds no prior identity to compare against. A control confirms the first observation after a resume reports nothing. The within-run and post-resume failures are two distinct mechanisms, not one mechanism seen twice.

**This is a third failure shape, and neither of the two in §5 covers it.** The mechanism is not absent, not unwired, and not inert: it is reachable, it is consumed, it is correct, and its output is dropped by a filter encoding a narrower notion of what may depend on the world than the runtime actually has. *Boundary: within-run for the discard, persist-resume for the empty invalidator.*

#### 4.9 RQ8 — Can the surviving anchor be compared at all?

OBSERVED Not always, and the runtime does not distinguish the two cases. The dependency tracker’s `observe()` silently skips any resource fact carrying no identity, and returns the same value it returns when nothing changed — documented as *the world did not move*. An identity-less fact does not even establish a baseline: it is skipped before the recording step, so a later observation that does carry an identity reads as *first seen* rather than *changed*.

The layer above refuses exactly this conflation. The revalidator keeps a third verdict for *could not check*, and its own documentation gives the reason: folding it into *it is fine* “is how a verification plane stops meaning anything”. **The runtime preserves the distinction in one module and destroys it in the module that feeds it.**

This path is production-reachable rather than hypothetical. Single-file reads carry a digest; *directory listings do not*, because a directory has no content digest — a listing of one tree produced three facts of which two carried identity, and at greater depth thirteen of which seven did.

A census over the receipts three mutating capabilities produce puts a boundary on the same question from the writing side rather than the reading side: `fs.write` yields two facts, both carrying identity; `fs.mkdir` yields one, carrying none; `fs.delete` yields one, carrying none. So identity cannot cover a directory, neither when observed nor when created.

**A correction we published before measuring the right thing.** An earlier version of this section drew a second conclusion from that census — that identity cannot cover a resource *after it is deleted*, and therefore that an approval whose effect is a deletion cannot be revalidated. **That is backwards, and the error was ours.** The census measures the facts a receipt carries, which is a statement about the world *after* the effect. Binding happens *before* it, while the file is still there. Measured directly, a deletion of an existing file binds an identity like any other write. What cannot bind is the deletion of a *directory*, and the reason is the directory, not the deletion. We record the correction rather than quietly amending the sentence, because a compressed restatement of a measurement — rather than the measurement — is what carried the error forward.

**What a surviving mutation does and does not say.** Batteries in this project answer a surviving mutation as one of *untested branch*, *weak assertion*, or *behaviourally identical for this test list*. The third is the one most easily misread. In one case here a guard survived mutation not because it is unreachable — it is live production code with two call sites — but because the list of tests handed to that battery contained no case that drives it. A battery is only ever meaningful for the tests it is given, and recording the answer as *dead code* would have erased exactly that.

#### 4.10 RQ9 — Is the approval that gets revalidated the one for this effect?

OBSERVED Not necessarily. Every result above concerns a premise that cannot be *established*. This one concerns a premise established over the *wrong effect*, and it sits upstream of all of them: the revalidation of RQ4–RQ8 runs on whichever approval the gate matched, and matching happens first.

The gate matches a recorded approval to an effect on equality of (`capability identifier`, `sorted resource list`). An effect’s resource list is mined from its parameters *by argument name*: the gate holds a fixed list of names that denote locations — `path`, `target`, `from`, `to`, `source`, `destination`, `file`, `dir`, `cwd` and a few more — plus a separate list for git refs. Values that survive canonicalisation become the effect’s resources. Two facts then compose:

1. A capability whose parameters name no location yields an **empty** list. `shell.run` declares `command`, `args` and `cwd`; only `cwd` is a location name, and its default `"."` canonicalises to nothing, because the workspace root is a container rather than a resource.

## 2. Two empty lists are equal.

So a single approval covers every effect of that capability. Measured through the real authority gate: an approval to run `npm test` dispatched `rm -rf src`, and an approval to commit “*fix the parser*” dispatched “*rewrite history*”. Both verdicts carried the source `user-decision` — not an escalation, not a default-allow, but the person’s own recorded answer applied to something they were never shown.

**The mismatch is not general.** A census over every write capability outside the filesystem, taken from each provider’s own capability description rather than from any restatement of it, gives six capabilities of which **four present exactly the resource the effect is about** (Table 2). It is concentrated in one family, and the census makes the reason structural rather than anecdotal: for a command, `command` and `args` *are* the effect, and they are not locations. The list is a list of locations, and it is complete as such.

**Reachability, measured rather than asserted.** `shell.run` is the clearest case, and `@mobius/env-shell` is depended on by nothing but itself: it is not composed into the daemon or into the run composition. **The shell arms are therefore a property of the gate, not a live hole in a shipped runtime**, and reporting them without that qualifier would describe a hole that does not exist. The repository provider *is* wired into the daemon, and `repo.commit.create` declares only a message, which harvests nothing — so the shipped instance of this shape is a commit rather than a command.

**A partial rescue that rests on prose.** The resource list does not begin with the parameters. It begins with the intent’s `target`, which is free text a model wrote, and a relative string is a legal path. Two commands described differently therefore carry different resource lists and do not match — which is why the substitutions above need the two descriptions to coincide, and one control arm confirms that a differing description blocks the match while another confirms that a shared one does not. The rescue is real and we report it as measured, but **it is not a safety property**: it holds only while a model happens to describe two different effects differently.

**The pinned invariant guards the other direction.** An existing regression pin states that a decision naming nothing authorises nothing. In that arm the effect *has* resources, so the two lists differ in length and no match was ever possible. The pin protects the side that cannot fail.

**Not predicted, and since repaired.** No pre-registration anticipated this. It is recorded in the ledger as unpredicted rather than as a refuted prediction, and the distinction is the subject of Appendix A. This is the one defect here that was fixed rather than reported: §7 gives the repair, what it deliberately does not reach, and the pre-registered prediction about it that failed. The arms above are retained at the commit that measured them.

**What this result may and may not be presented as, given concurrent work.** The divergence between an approved effect and an executed one is named in the recent literature, and a mechanism for it has been formalised and benchmarked. One line names *consent integrity* [36] — the action shown to a person must be rendered by a trusted mediator from the real action and bound to the exact action that executes — and its taxonomy already includes approval for one action followed by execution of another. A second defines a canonical action identity, fingerprints it, and binds approvals and receipts to that fingerprint rather than to raw text, evaluating an approval-binding dimension across a large variant benchmark [5]. **We therefore do not present this as an**

unaddressed problem, and a canonical-identity mechanism of that kind would address the matching failure reported here.

**Two things survive, and they are what we claim.** First, the consent-integrity setting assumes an adversary who controls the agent and forges the narration; **here there is no adversary, the recorded decision is truthful, and no component is compromised** — the approval matches a different effect because arguments are excluded from the identity by design and two effects naming no location canonicalise to the same empty list. The property is violated by a *match* rather than by a lie. Second, and this was our reading until the user’s ruling: **that the coarse key was not the defect**, because its argument was correct and what failed was an unstated premise beneath it — that the resource list names *what the effect is about*. The repair that holds addresses that premise (§7). **The ruling reverses the first half:** an ordinary approval covers the concrete proposal the person saw, so a key without the proposal’s content is itself a defect, and §7.3 measures it on the shipped daemon for an effect that names a resource.

## 5 Root-cause analysis

The verdicts are the same and the causes are not (Figure 6). Branch A: no precondition, no forced look, no grounding, no basis, no comparison. Branch B: a premise is declared, a basis exists, the foreign mutation moves no revision, drift is zero, and the guard returns before the mode switch. What they share is a pattern rather than an input: every gate keys on something a foreign change cannot touch — a counter it cannot move, or a premise that was never built.

**A cause one plane above, and not an instance of that pattern.** RQ9 does not fit the sentence above, and we keep it separate rather than widening the sentence until it does. Branches A and B are both accounts of a premise that cannot be *established*. The matching defect is an account of a premise established over the *wrong effect*: the gate keys on a resource list, the list is empty, and equality on emptiness is a match. Every defence built in this line — revalidation, binding, coverage — runs *after* that match, so correctly revalidating a wrongly matched approval changes nothing. This is why it is reported as a separate result and not as a third branch: the two branches answer *why was nothing compared*, and this one answers *what was the comparison about*.

**Neither branch rests on one experiment.** Branch A is measured in four independent experiments and branch B in eight, and each spans all three boundaries — a single run, a run boundary, and a persistence-and-resume cycle. These are counts of experiments rather than of arms, derived from the ledger; the arms themselves are in Figure 4, which is generated from the same rows. **We say so because the rest of this paper reports results measured once**, and the difference between a result seen four times across three boundaries and a result seen once is not visible in a verdict.

**Two recurring shapes, kept apart.** *Absence read as a value — I could not tell* recorded as *nothing moved* — has been measured four times in this system, each time somewhere new: in a provider’s verdict, where a fact carrying no identity was returned as *the world did not move*; in a runtime fact, where a withdrawn design would have recorded *absent* as an observed value; in an authority matching key, where an empty resource list compared equal to another empty one; and in the loop written to repair that third one, whose first version reported a resource whose covering look had *failed* as a resource *nobody asked about*. That fourth instance was caught by an earlier experiment’s pinned arm going red.

**A fifth was held out of this count and has since joined it.** The criterion evaluator folds *could not be evaluated* into *not satisfied*, and when we first reported it we had found it by reading the

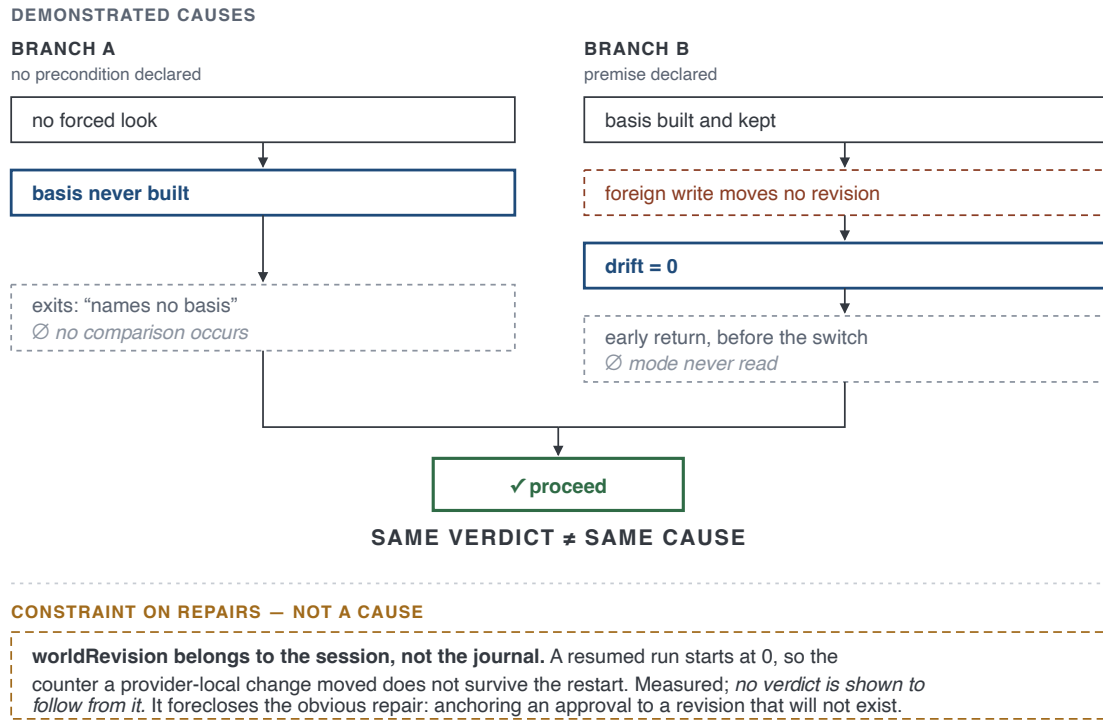
source while the other four were each an arm in the ledger. It was stated where it bore on the argument and excluded from the tally, on the grounds that mixing a read into a count of measurements would do to this list what merging two shapes did to the earlier one. It is now measured in four arms of its own and is counted here. **We record the exclusion and its lapse rather than quietly renumbering**, because the reason for holding it out was conditional and the condition expired — which is the only way to tell that the rule was doing work rather than decorating the paragraph. **It is also the sharpest of the five**: the collapse occurs in the same package as the file whose opening lines forbid it, and that file’s own fold obeys the rule it states.

A second shape recurs and is not the same failure: *declared and unconsumed* — a plane that exists, is computed, and nothing reads — and it has appeared in three progressively worse forms. A lineage of unwired seams, already six deep at an earlier checkpoint, was *declared without consumers*. The two surviving mutations of §7 describe a mechanism that could have been *declared, wired, and never produced a single value*. And a flag computed on every look for many versions, whose only four readers are its own tests, turned out on measurement to be *named for a quantity it does not compute* — the four tests asserting on its name each took the name for its meaning (§7). The provenance of §4’s third durable decision belongs to this shape too, and is its quietest instance: the fields are present, correct, and stamped by every provider that mints evidence, and the path that reuses that evidence reads none of them.

**We report these as two observations about this system, not as laws**, and what would refute either is a plane of this runtime where the guards built for the previous instance do catch it. A further instance of the first shape occurred in our own reproducibility checker and is *excluded* from the count for that reason — it is reported in Appendix A, because counting tooling alongside the system under study would leave the refutation condition above with no determinate scope. Keeping them apart is load-bearing rather than tidy: merging them inflates both counts and leaves neither falsifiable. **An earlier version of this paragraph did exactly that**, listing a coverage gap among the first shape’s instances — the experiment that found it had in fact *distinguished not asked* from *asked and failed* rather than collapsing them, and belongs to neither list. The count survived the correction and the membership did not, which is the failure mode this paper reports elsewhere in its own reasoning (Appendix A).

**A measured fact that is not a third cause.** A resumed run opens a new session at revision zero, so the counter a provider-local change moved does not survive the resume. We state this as measured and deliberately keep it out of the causal analysis: no arm shows a verdict following from it, and the positive reason is that the resumed run *re-looks*. The previous run’s observation is *unevaluable* because its session is gone, so the gate declines to reuse it, and the fresh look builds a basis at the current revision against a world that already contains the earlier change. The counter went to zero and re-observation covered it. In the premise path a vanished session is *caught* rather than exploited (§4, RQ5); nothing in this runtime carries a basis across a resume to compare wrongly; and the verdict in the arm that exhibits the reset comes from branch A. Its consequence is for §6, not here.

The V60 contrast is stated at the representation level and not promoted further: an observation-derived premise records session, revision and evidence, and can cease to rule; a `UserDecision` records capability, resources and a timestamp that is never compared against a clock, and continues to match. **That a durable permission needs some means of being withdrawn is not a new observation**: capability systems have carried the question since they were named [15, 16], and reviewing and revoking privileges already distributed was given a treatment forty-seven years ago [17]. What this contrast adds is not the need for revocation but its *trigger*: both records here are revocable in principle, and the one that ceases to rule does so because it names something that can become unavailable, while the one that does not names nothing that could.



**Figure 6.** Two demonstrated causes reaching one verdict, and — shown separately, below the rule — a measured fact that is *not* a cause. Where no precondition is declared, no basis is built and no comparison occurs; where one is declared, the comparison happens and returns zero because a foreign write moves no counter. The counter reset across a resume is measured, but no verdict is shown to follow from it; it appears here only because it constrains the repair those two causes invite.

## 6 Design implications

**CANDIDATE DESIGN — NOT YET VALIDATED.** One repair is ruled out by measurement rather than by preference. Anchoring an approval to a world revision cannot work across a resume, and the reason is stronger than the foreign-writer case that motivates it: a mechanism comparing revisions across a resume cannot see a change *the runtime itself made in the previous pass*, because the session that owned that counter no longer exists. A same-world control adds a second constraint — resume alone does not invalidate an approval, so a repair may not simply expire every approval at every reload.

**What measurement has eliminated.** Four anchors could in principle carry an approval’s validity across time; three are ruled out by the measurements above rather than by preference. A *revision* cannot: the counter does not survive the resume that would need it. A *session* cannot: anchoring to one would invalidate every approval at every resume, which a same-world control measured must not happen. The *runtime’s existing identity memory* cannot: it is task-keyed and per-run, which is exactly why it discards the detection it already makes (§4, RQ7). What remains standing is a *resource identity recorded on the approval itself*, which converges with the architecture record’s independent conclusion that identity means digests rather than counters.

We report this as an elimination result, not a design — and the surviving anchor arrives with measured constraints rather than open questions. Any identity-anchored check must distinguish *the identity is the same* from *there is no identity to compare*, because the plane it would build on returns one value for both (§4, RQ8). It has a measured coverage boundary, reported in §7 now that the

mechanism exists to measure rather than predict.

**The largest gap is neither of those.** An approval can be bound to a resource identity only if an observation was taken at approval time for that identity to be derived from, and one is taken only when the proposing intent declared a premise. **What is carried forward is the derived value, not the observation:** the binding also records a link back to the observation it came from, and §7 reports that nothing reads it. That is RQ1, and it is the whole of the mechanism: no declaration, no forced look, no grounding, nothing to bind. Nothing in this runtime requires a proposer to declare one, and the arms in which none was declared dispatch with no basis at all.

**So on the runtime as it stands, and absent a decision to spend an observation when a person is asked, identity-based binding is structurally unavailable for any approval whose proposing intent establishes no observation.** Coverage is gated by premise establishment, and nothing requires a proposer to establish a premise.

**That is a statement about which approvals are eligible, not about how many.** We measured no workload and therefore say nothing about how often proposers declare a premise in practice; the arm composition of this study is experimental design, not a frequency in a running system (Appendix A). What the measurement supports is the gate itself: where no premise is established there is nothing to bind to, at whatever rate that occurs.

This gate sits *before* the coverage boundaries of §7 rather than alongside them. Those decide which observed resources can bind; this one decides whether an observation exists to bind to at all. No statement about what identity-based revalidation covers is complete without it.

Both conditions in that sentence are load-bearing, and it is a statement about coverage rather than a verdict on the anchor. The anchor is not refuted; the decision below is exactly what would make premise establishment routine rather than incidental. We put this in the design section rather than among the limitations for that reason.

**Where a design would choose rather than measure.** A design for this anchor exists on record, and has since been built — §7 reports what that produced, and it is not mainly the mechanism. Its semantics were complete on paper: a three-valued verdict reusing the vocabulary the runtime already applies to observation revalidation, in which both non-reuse outcomes escalate to a person rather than refuse — recording *the person was never asked* as *the person said no* would be the same conflation this paper reports one layer down (§4, RQ8). What blocks implementation is not evidence. Two questions are open and neither is answerable by measurement: what to do with approvals already recorded without a binding, and whether to spend an observation at escalation time so that there is something to bind at all. The second is the precondition for the anchor covering anything — the gap above, arriving as a price. The framework for that price is already written: an earlier decision record settles how such a check is counted and charged, states that it authorises no spending of its own, and names the constraints that *the first slice making an action carry a premise* inherits before it chooses a shape. So the open question is the price, not the framework, and not permission.

It is worth saying why that price is not easily avoided by bookkeeping. The obvious escape is to charge an observation taken for authorisation to something other than the user's goal. The runtime's budget contract is written *purpose-blind* — defined as a count of observation events rather than of observations of a given kind — so any new use falls inside it by construction, and exempting one means amending a stated contract rather than filling an undecided blank. That contract has been tested at least once: a verification look was found to have reached a provider while remaining free and invisible, and this was treated as a defect and repaired by routing all accounting through a single function. A look taken to verify is still a look.

Escape routes do exist, and each meets a decision this runtime has already taken: not accounting



for the look at all would create a channel of free reconnaissance, since a binding requires the result to enter the journal; and a separate allowance is a second budget kind, which this cost model explicitly refused. Those are decisions and can be revisited. But as things stand the choice falls between paying for the binding and not having one.

Where version comparison cannot fire, the trigger classes in use elsewhere are lease expiry, writer-initiated invalidation, read-triggered repair, and background anti-entropy. Two are ruled out here by measurement rather than preference, and the remainder assume a check is a message where here it is an observation charged to a budget.

Two corrections to the price, both from building it. First, a look here is model-formulated, so the binding costs one observation *and* at most one model call — at most, because the observation source caps model calls and falls back to a direct look once the cap is reached. Second, and larger: one observation per binding was the accepted figure until looks were made to *cover* what a binding needs, and the worst case is now **one observation per addressable resource**, bounded by the resources the person is being asked about. **Reuse still costs nothing.**

**That last sentence is false and we measured it to be false.** It was the half of the cost claim this paper leaned on, it was pre-registered as the half most likely to be wrong, and its failure was pre-registered as the point at which to stop. **Reuse buys the same covering look that first use buys:** at eight addressable resources a reuse spends eight observations, exactly as the first use does, and the figure tracks the resource count rather than staying flat in it. **The correction is the fifth on this line and, like the four before it, it runs upward** — *costs nothing* becomes *costs what first use costs*. The direction was itself a pre-registered prediction, so the pattern was tested rather than admired.

**What survives is a bracket, and both of its endpoints are real provider behaviour rather than fixture settings.** With narrow looks the worst case above is realised and tight: one observation per addressable resource. With a directory listing that carries a digest per child entry, **one look serves any number of them. So the runtime contributes the covering loop, and the multiplier from resources to rounds belongs to the observation source and the provider.** A measurement that had fixed one look strategy would have reported a fixture property as a runtime cost; neither endpoint alone is the answer.

**The inputs were counterfactual, and a check made for another experiment found it.** Every arm proposed `fs.write` with an extra parameter carrying the additional resources. The capability's schema does not declare it and admits nothing undeclared, and the shipped compiler refuses such an intent before the gate — the assembled runtime hands the compiler each capability's schema, and nothing turns enforcement off. So the code measured is shipped, and the channel that put  $N$  resources on one write is one this runtime does not produce. At  $N = 1$  the input is equivalent to the shipped form, which is an argument rather than a measurement, and a replication in the shipped mode is pre-registered; no shipped-mode measurement exists for  $N \geq 2$ . The refutation of *reuse still costs nothing* stands in substance at  $N = 1$ , and is counterfactual evidence until that replication runs.

**Two bounds travel with every number here.** No arm here drives the runtime's model port or makes a network call (§9), so **the cost that would dominate in production cannot be measured at all** and what is reported is a count of observations, not a latency. Wall-clock figures were recorded and are context only; they are not production latency and are not quoted as such. And the attribution rests on a second baseline rather than the first one tried: removing the mechanism that *consumes* identities leaves the full cost in place, because the look is bought before that consumer runs. **A correctness baseline is not a cost baseline**, and using the one for the other would have reported this mechanism as free.

That second correction also retired an argument we had accepted. The claim that a binding is *bought once* was carried over from a decision record whose rule reads *at most one check per basis, and none where the answer cannot change the outcome*, memoised per observation. It does not

apply: a look that never covered a resource **never asked about it**, so a second look is not the same question repeated, and the rule's own second clause argues for taking it — a covering look does change the outcome, turning *nothing asked* into an identity, which is the whole difference between one-time consent and a re-checkable one. **The rule bounds repeat purchase across reuse, and that bound still holds**; it never required giving up after one look.

## 7 What building it revealed

The surviving anchor was implemented: an approval now records the identity of each resource it was granted over, taken through the loop's single gated door, and is revalidated before reuse. A changed identity escalates rather than dispatching, and *I cannot tell* stays distinct from *it changed* — the conflation §4 reports one layer down. The control that constrained the design holds: a resume alone does not invalidate an approval.

**That was one point, not a surface, and the difference has since been measured.** The arms above measure what the repair binds and what it refuses; they did not measure it against the runtime it replaced, so nothing established that it fails to re-escalate approvals that should still have been reused. **A mechanism that invalidated everything would have passed every arm in this section except that one.**

That comparison now exists. Twenty-six scenarios were run in two columns — the built runtime, and the same tree with one mechanism removed at a single site — with each scenario's correct outcome labelled *before* measurement as a reuse that ought to stand, one that ought not to, or an escalation on uncertainty whose rightness this paper does not settle. The last is reported in its own row, because folding it into over-invalidation overstates the harm and folding it into intended correction flatters the repair. Across the set, **all fourteen reuses labelled *ought to stand still dispatch*, and nothing moves in the permissive direction.** A mechanism that invalidated everything would now fail fourteen. **Fifteen of the twenty-two stand and fall cells hand the gate a write the shipped checker refuses** (a path without its content). For nine, outcome, rule and route are unchanged once content is added **in both columns**, and the baseline runs reproduce the ledger's column 26/26; six rest on inputs no compile produces. The five loop arms and two commits compile as shipped. §7.4 measures the top-level case on the shipped daemon.

**That mechanism is not, however, uniformly worse, and reporting both directions is what makes the difference visible.** It fails those fourteen, and it is *right* about the one cell in the set that both repairs get wrong: an effect whose only addressable resource is the model's free-text target ought not to be reused, and is dispatched in *both* columns, because one repair's branch requires the resource list to be empty and the other's requires it to be addressable, and this effect satisfies neither. A mechanism that refused everything would escalate it correctly. So the comparison does not license *the repair dominates*; it licenses a statement about which cells each gets right, and the strawman wins one of them.

**The zero is a measurement rather than a blind spot, and that is the part that took the work.** A mutation making revalidation report an unchanged resource as changed — over-invalidation, injected — fails eight of the suite's twenty-nine cases. A cell in this set can therefore register the thing the comparison reports not finding. **Seven of the eight are cells that existed before the same-world control was added**, so that control extends the check into a world where nothing changed rather than being what makes the zero readable; we state it that way because the stronger reading was available and is wrong.

**The repair turns on which resource moved, not on who moved it.** Four end-to-end arms cross *who wrote* against *what was written*: a change this run made through the coordinator, whose session counter can see it, against a foreign write behind the runtime's back that moves no counter; and each against the approval's own resource or an unrelated one. Both writes to the approval's own resource escalate — with the identical recorded reason, *this was approved, and changed*

*since approval*” naming the file — and both baselines dispatch. Both writes elsewhere are reused in both columns. The foreign cell is §4’s anomaly with the repair in place, at a persistence-and-resume boundary.

**What this does not license, stated because the numbers above are easy to read as more.** Twenty-six cells over one gate and one filesystem provider is not a validation. *Foreign* here means outside the coordinator, *not* outside the process: the write is a real filesystem write nothing in the runtime recorded, made from the test’s own process. A second operating-system process, a concurrent writer, and **a second provider over one root — the cross-provider case §4 leads with** — are unmeasured. An approval whose proposing intent established no premise has no arm, so a prediction this paper made about it cannot be refuted. The overhead such a comparison would need to price is left to a separate measurement and is not reported here as an empty result.

**One limit is of a different kind and belongs beside those rather than among them.** Everything above bounds what was *measured*; this bounds what the mechanism can *reach*. The anchor is resource identity, and identity simply *is* the content digest: a resource fact carries one when its payload carries a digest, and not otherwise. **A directory’s own payload cannot carry one**, and the reason is not an omission — a digest is computed over bytes, and a directory has none. The runtime then keeps only facts whose identity is defined, so a directory’s current identity is undefined by construction, and an approval over a directory path is bound with an absence reason in place of an identity.

**Two things must be said with it, or it misleads in both directions.** First, the neighbouring case is weaker: a *file* listed inside a directory carries a digest only when the listing was asked to hash its entries, which is a flag rather than a limit — so *identity is sometimes absent* is a configuration for files and a structural fact only for directories. **And the limit is stable under nesting:** a subdirectory appearing as an entry of a listed parent carries no digest either, at any depth and whichever way that flag is set, so listing a parent does not make a nested directory revalidatable through its entry. That follows from the same fact rather than from an enumeration of the places digests are produced — every one of them computes over bytes read from a file, and there is no route by which a directory acquires any. Second, and more important, **the degradation is toward refusal, not toward permission:** an approval with no identity to compare takes the no-identity branch, which treats the person’s answer as covering the use it was given for and no later one — dispatch on the first use, and an *unevaluable* refusal naming the missing identity on any later one. Saying only that such an approval *cannot be revalidated* would invite the opposite reading.

**The repair carries a field that nothing reads, and we found it by looking for it.** A binding records the resource, the identity as at approval, an absence reason when there is none — and a link back to the observation that identity was derived from. **That link is produced, carried into the effect, persisted into the durable decision, loaded back on resume, and then not read.** Six arms through the real loop, coordinator, journal and notification adapter differ only in that field — present with the runtime’s own value, omitted entirely, and pointing at an observation that does not exist — and **every observable is identical in both world states:** the stop reason, whether a question is raised, the authority events, the observations spent, the projection, the sentence a person is shown, and the revalidation verdict. The pre-registered falsifier was any observable differing; it fired zero times. **The arm pointing at a non-existent observation is the sharpest: a field that is merely carried cannot notice a broken link, and a field that is resolved must.**

We report this as *not reachable* rather than *absent*, because every stage before the last one works. **And we report it against ourselves.** The design rationale for that field was that a durable approval should be able to name what the person was actually shown; the measurement says the mechanism binds to a *value* recorded at approval time and to nothing else. It is the paper’s own recurring shape — declared, computed, and unread — occurring for the fifth time, in the repair this

paper proposes rather than in the runtime it examines.

**Evidence class, stated because this is the one limit in this section resting on reading rather than on an arm.** The chain from a directory to that branch is verified at five sites across four files. The branch's own behaviour is measured, by the two arms that reach it through a file whose binding records the same absence reason. No arm in the comparison has a directory as its approved resource, and we have not manufactured one to decorate a bound. **So the repair is reported as implemented, and compared against the unrepaired runtime on a bounded set over which it does not over-invalidate — not as validated.**

**The mechanism is the small part of the result.** Its design was written down and reviewed before any of it was built, and that review is on record. Building it contradicted the design in four places, and none of the four is a coding error. In each, the design held a belief about how this runtime *represents* something — what a grant covers, what a resource list means, when an approval is consumed, what a look reports — and the built runtime represents it otherwise. The procedures were written correctly against those beliefs, which is why **static review surfaced none of the four**: a review reads procedures, and each of these is a property of a representation. We state them as four contradictions, the design's claim against the measurement.

**Contradiction 1 — authority and evidence are not independent planes.** *The design assumed* that deciding what an effect may touch and observing what it is about are separate concerns, so the binding could be specified without saying anything about grants. *Measured*: the shipped envelope granted nothing, so the runtime could not observe what it was asking about. The approval therefore recorded nothing, its reuse was unevaluable, and the failure did not present as a refusal anyone would notice: it presented as *a person answering an approval and being asked the identical question again* — two approval requests for one write. A runtime required to identify what an approval is granted over must be permitted to observe it, and “grant nothing” is not a safe default for such a runtime.

**Contradiction 2 — one resource list cannot carry two semantics.** *The design assumed* that the resources an effect touches are one set, so permission and binding could range over one list. *Measured*: they are two sets with opposite failure directions. Binding initially ranged over the same resource list that decides permission. That list begins with free text a model wrote — in the measured case, `["notes.txt in the workspace root", "notes.txt"]`. Over-inclusion is correct for a fail-closed authority scope and fatal for binding: the prose entry is unidentifiable, and a correct binding of the real path was reported unevaluable because of it. Permission and binding now range over different lists. **This is a separation of semantics, not a data-cleaning fix**: the prose entry is not dirty data to be filtered but exactly right for a fail-closed scope and unusable for an identity, and no single list can be both.

**Contradiction 3 — consumption is a fact about a dispatch, not about a run.** *The design assumed* that “a later use” is expressible as a run boundary. *Measured*: it is not. The implementation read “later” as crossing a run boundary, and an approval is always consumed in a run after the one that asked, so **the first use was already a later one and the single-use rule was a zero-use rule**. The interesting part is not the off-by-one: it is that *which* approval permitted *which* dispatch is not derivable from run structure at all, and belongs on the authority record of the dispatch itself. Finding this required asking that question of the journal, which exposed a second representation gap — **the event recording a permitted action carried no authority at all, while refusals and escalations carried a full authority report**. The system could say in detail why it had declined and nothing about why it had proceeded.

**Contradiction 4 — *not reported is not absent, and this is the sharpest.*** *The design assumed that a look which says nothing about a path establishes that the path is not there, which would let approvals over resources yet to be created bind. Measured false:* a look reports what it looked at, and says nothing whatever about a resource it did not examine. The inference from silence to absence requires the look to have been complete over the request, and nothing establishes that. That is the conflation of §4 — *could not check* folded into *it is fine* — reintroduced inside the mechanism written to refuse it.

**What makes it the sharpest is what did not catch it.** A pre-existing test had ruled on this exact failure mode in principle, one layer down and against a different function, in words that describe it precisely. It did not go red. The defect was restored deliberately and measured: the slice’s full set of pins — 44 tests — stayed green, and so did its seven-mutation battery. The reason is worth recording, because it generalises past this bug: every arm supplied a look that happened to cover its own target, so the faulty branch was never reached, and whether it is reached depends on how deep the path is. A listing of the root directory incidentally covers a file in the root and can never cover a file two directories down. **A fixture suite written at the root agrees with this bug.** What found it was neither the test nor the battery but printing the value: a file just written to disk was seen to bind as absent. An arm using a deliberately target-ignoring port and a nested path was added afterwards, and the battery gained an eighth mutation to convict it.

**The instruments both succeeded and failed here, and reporting only the success would be selection within one slice’s own evidence.** Two mutations survived the first battery run, both of one shape: the mechanism would have been *declared, wired, and inert* — never producing a single identity — while every pin stayed green, because an approval bound without an identity still works once. That is §5’s subject reappearing in the repair for it, caught by an instrument rather than by review. A second test exists to convict those mutations, and one of its own arms initially had the same defect.

That is the positive case. The negative one is the paragraph above: on the withdrawn `ABSENT` reading, the green suite and the passing battery *both agreed with the defect*. The same slice therefore supplies an instance of an instrument finding what nobody was looking for and an instance of the whole instrument endorsing an error, and we report both. Across this line, surviving mutations have caught several weak assertions and reading has caught others; we have no basis for ranking the two, and the count is small enough that we do not try.

**Bounds recorded rather than routed around.** Absence cannot be bound, because the provider reports a missing path as a failure that only a namespaced code distinguishes from an unrecognised error, so approvals over creation remain single-use. An effect whose parameters name no resource records no binding at all and is reused without revalidation, as before, for identical canonical arguments.

## 7.1 How far the binding actually reaches

The two surviving mutations above name a specific risk: an approval bound without an identity still works once, so a mechanism that never produces a single identity passes every test that does not check for one. That risk points at the mechanism itself, so the next measurement was aimed there — at presence versus effect rather than at presence alone.

**As the runtime stood when this was measured, of the five mutating capabilities that escalate, three could carry a revalidatable approval, and only when the target was a file that already exists.** The bound was later moved by a repair, reported at the end of this section; the measurement is retained at its own commit rather than rewritten, because what it bounded is what the design in §6 was reasoning about. `fs.write`, `fs.patch` and `fs.delete` bind. `fs.patch`’s arm

used flat arguments the shipped compiler refuses, which makes it counterfactual; the path it binds is the same, but that is an argument, not a measurement. `fs.mkdir` never does. `fs.move` never does. `fs.delete` over a directory does not. Outside that range an approval remains an honest one-time consent: a real mechanism, and a much smaller one than its name suggests. Every claim in §6 about what this anchor covers must travel with that condition.

**Three causes, not one.** The cases that cannot bind do not share a reason, and each is reported with its own:

- *the resource does not exist yet* — a creating write, or a directory being made;
- *it exists, was observed, and the fact carries no identity* — deleting a directory, which is §4's directory finding arriving through the binding path;
- *nothing ever asked about it* — the destination of a move.

Two earlier generalisations each turn out to be one third of this: *directories are the general case*, and *creation is the class*. Each is true of its own row and neither covers the others.

**What was not predicted, and it is the useful part.** The pre-registration reasoned about how bindings are combined and never asked *what feeds them*. Two arms settle it. Creating the destination of a move first, so that it exists, still does not bind — so the cause is coverage, not existence. Supplying the same effect with a single look that covers both resources binds *both*.

So a multi-resource effect is not structurally unbindable. One observation can carry facts about several resources. What is missing is that nothing selects a look that *covers* the resources a binding needs, and it gets one attempt.

**The defect is at the seam, not in any provider — and it is not that the instruction is wrong.** The instruction says both things. One line of the system prompt requires that a request's parameters *mention* the resources in doubt; a line of the user prompt, built for the same call, requires that they *cover* them. **The prompt contradicts itself, and the check implements one half:** it tests the request's parameters for the target as a substring, under a local name that says so. Binding needs coverage, and coverage is the half not enforced. The measured consequence is uncomfortable and exact: *a port that obeys the enforced half binds worse than a port that ignores it*.

**A flag that was never about what its name said.** Every look this runtime takes computes a flag whose name claimed it reported coverage, and has done since the version that introduced the observation port. The flag has four readers and **every one of them is its own test**; a related trail on the observation source has none at all — computed afresh on every look, for many versions, and never read by production code.

**Measuring what it actually computes made it worse than unconsumed.** It reads the *request* and never the result, so it reports what was asked for rather than what was seen. Three arms fix the consequences in all three directions: a request naming a file reports *covered* and the look then fails to find it; a request naming nothing reports *not covered* and the listing returns the target anyway; and because the test is a substring, a request for one path reports a different path as covered. **The look this flag calls uncovering is the one that covers** — which is the mechanism behind the obedient port binding worse, and it is the same arm reported above. The flag was renamed to what it computes rather than given a consumer, and the option of wiring it was retired rather than deferred: it was never the fact its callers needed.

This is the second recurring shape of §5 in a third form. Not merely declared without consumers, and not merely computed and unread, but **named for a quantity it does not compute** — with four tests asserting on the name, each taking the name for its meaning. **We left the prompt's**

**contradiction in place.** Half of it is right, but choosing which half changes what the model is asked to do, and no arm in this study drove a model; picking one by argument alone is the move its own pre-registration exists to refuse.

**The bound moved, and one earlier arm reverses.** Selecting looks that *cover* what a binding needs was then implemented. Measured after the change: a move whose destination exists binds *both* of its resources, reversing the arm above that established the coverage cause. The earlier arms stand at their own commit and the new ones are recorded separately, so the table of what binds is a statement about a runtime version rather than about the mechanism in general. **Building that repair produced one more instance of the first recurring shape:** its first version reported a resource whose covering look had *failed* as a resource nobody had asked about — the same conflation, inside the loop written to repair the previous one. What caught it was an arm pinned by the earlier experiment going red, not a review of the new code.

**PENDING. Partly measured. This block is kept, with its original preregistration intact, to record which of its own rows now exist and which do not.** The shape was fixed before the measurement so that the table could not be assembled selectively once it arrived. It named four rows — same world resumed, foreign write between approval and reuse, approval over a resource that never carried an identity, approval whose proposing intent established no premise — and required both failure directions, so that a repair blocking every reuse could not score perfectly on one of them.

**Two rows are now measured and are reported in §7:** the foreign write, at a persistence-and-resume boundary; and the approval over a resource that never carried an identity, within a single run, where the reuse is refused as unevaluable on the ground that the resource had no identity when it was approved.

**A third is now measured too.** *Same world resumed* was, when this block was last revised, present only at a run boundary — lifting the original control onto the gate had moved it off the boundary at which the anomaly was demonstrated, and every arm that did resume differed from a same-world control by a write somewhere. A control was added for it: approve, resume, write nothing, reuse. The approval is reused, in both columns. It is the only cell in which nothing changed, which is why the arms that write to an unrelated resource are not substitutes for it — each of those supports *something changed elsewhere* and the approval was reused, which is a different sentence.

**One row remains unmeasured.** *Approval whose proposing intent established no premise* has no arm at all, so the prediction this paper records about it — that the mechanism cannot apply there — remains unrefutable rather than confirmed.

**Also outstanding:** the cross-provider case, where the change arrives through the second provider whose writes the revision counter cannot see, which is the anomaly §4 leads with and is not the same object as a foreign write from within one process; and *observation overhead*, which is a separate measurement not yet taken rather than a column that came back empty. This block stays until those exist.

## 7.2 A matching key that was empty rather than wrong

The defect of RQ9 was repaired, and the repair is narrow on purpose. Where an effect names a resource the coarse key is untouched. Where it names none, a discriminator decides: the canonical form of the compiled action's arguments, which already existed for an unrelated purpose and is already pinned, so the finer key is not invented here. It is an optional field on the effect and on the recorded decision, set only when the resource list is empty, and deliberately kept *out* of that list — entries there reach the scope polarity rules, the resource prefixes, and the sentence a person reads, none of which take anything but a path.

**The coarse key, and a ruling that it is a defect.** The argument for that key is on record: authority is about what class of thing may be done to what, and a person approving a write to a named file approves it *as an effect* rather than as one occurrence of it. **The user has since ruled that argument wrong:** an ordinary approval is an instance approval of the proposal the person saw, and any wider

grant must be a separate, explicit mode (§7.3). The argument also rests on a premise it does not state — that the resource list names the *to what*. With an empty list there is none, the class collapses to everything the capability can do, and **the comparison is not wrong, it is empty**. The same discriminator was needed in a second place for the same reason: two effects naming no resources produced the same question identifier, so an answer to one dismissed the other.

**Bounding is not repair.** The comfortable answer is that a use limit bounds the damage to one use. **None applied.** The single-use rule lives in the branch for a binding that names a resource without an identity, and an effect naming nothing never reaches it: the reuse path returns before revalidation when nothing is addressable. The run-scoped repetition guard did not apply either, because the wrong effect was a different command. Both are read from source rather than measured. Before the repair, then, one approval matched every command of the capability (measured, RQ9), and nothing limited how often. **There was no bound to mistake for a repair** — and a one-use bound would not have been one, because the wrong effect can *be* the first use: in the measured arm the recursive delete was dispatched under a decision that had never been used at all. **Bounding an unbounded match to one wrong occurrence is not a repair.**

**A prediction that failed, and an arm that stayed green for a different reason.** The pre-registration expected that after the repair two commands sharing one description would be told apart by their arguments, so the rescue that rests on prose would stop carrying weight. It does not. Two commands sharing a description each name a resource — the description itself — so they take the coarse key and still match each other. **The repair addresses effects that name nothing, not effects a model gave the same name.** The arm is retained as refuted with its wording unchanged; rewritten, the prediction would read as confirmed while describing behaviour that never occurred, and the prose rescue is still load-bearing where it was.

**A repair no wider than the defect, stated so it could fail.** The claim that the change does not reach effects that name a resource was pre-registered against a specific outcome: if any pinned invariant about such effects moved, the change is wider than described and the fault is the design's, not the pin's. Seven such pins across six experiments stayed green and unmodified.

**Two survivors, two different answers.** The battery produced one of each kind. The first is an *untested branch*: a decision recorded before the field existed carries no discriminator, and no test contained such a decision. The answer is the one an earlier slice reached one layer down — it matches nothing rather than everything, so the affected set is bounded and shrinking and the person is asked again rather than overruled. The second is *behaviourally identical for the tests given*: attaching the discriminator to every question identifier changed no test, because the gate sets it only when the resource list is empty and that branch is unreachable for any effect this runtime produces. The guard was kept rather than deleted, because the question-identifier function is exported from the protocol, accepts a plain object, and nothing can require the gate to be its only caller. It is now pinned directly instead of only through the gate.

### 7.3 An approval covers a question, not the proposal the person saw

OBSERVED Measured on the shipped daemon (its binary, its socket, a real client answering, no decision seeded) in three arms, after the user ruled that an ordinary approval is an *instance* approval of the concrete proposal the person saw, and that any wider grant must be a separate, explicit mode. The approval request shows the capability, the resources (the path and the model's prose target) and a fingerprint of the file, **and no content in any arm**. Its identifier is the `(capability,`



resources) key the match uses, so two proposals for one resource are one question. With the world unchanged, a re-proposal that kept the target and changed the content **ran on the earlier approval**: the dispatch names that decision, revalidation compared the world, found it unchanged and said *reusable*, and the new bytes are on disk. Re-proposing the same content dispatched it; a proposal naming another file was asked as a new question. All three outcomes were predicted from source before any arm ran; what the source could not supply was what the person was shown. **So the person approved neither proposal's content**: the first was never shown, and the second was never asked about. Instance approval therefore needs the request to *show* the proposal as well as the match to *compare* it, which is a decision the user has not yet taken and nothing implements. *Bounds: the re-proposal was scripted; one capability, one resource; how often a model re-proposes differently after an answer is not measured.*

#### 7.4 On the shipped daemon: E2E-1

REPLICATED Fourteen arms through the shipped binary, its socket and a real client answering, no decision seeded, each against a column with the re-judge removed at one site. The citable run persists every daemon journal, the harness's clock, pids and world digests, the stub's prompts and replies, and the extractor's trace, with SHA-256 manifests. **Every file a manifest hashes is carried in the repository, and all eleven verify from a checkout of tracked files alone** — 440 entries, checked from a clean checkout rather than from the tree that wrote them. For most of this work they did not: the ignore rule excluded the journals the manifests hash, so six directories verified only on the machine that produced them while these results cited them, and the rule was changed once that was measured rather than assumed. It was replicated five times for HEAD and four for each other column, with identical gates and final worlds, and it agrees on every earlier field with the first anchored run, whose journals were lost with their temporary roots. **On an approved top-level file, a foreign write between request and answer, or between answer and dispatch, is refused by the gate, and the column without the re-judge executes the stale write**; an unchanged world, an unrelated file and a write made before the request dispatch in both. This is the anomaly and its repair, reproduced on the path a client takes. **A nested target that the model's look did not name executes stale over the foreign write, in both columns**: the look fell back to the root, the target bound without an identity, and its one use went unchecked. The same arm on a top-level file is refused, so the boundary is nesting, not the ignoring. The patch workload was first not reached: its plain-words intent shared no term with any capability, and the router abandoned it. After a routing change let arguments reach the one reversible capability that accepts them when the sentence reaches none, its three arms reached the gate and matched their registered predictions in both columns: with the re-judge the relevant and the irrelevant foreign edit both escalate, the second unnecessarily; without it the provider's exact match refuses the relevant edit and the irrelevant one executes correctly. The patch result below reproduces on this path. The daemon serves no resume: across a restart, a person's answer to the old request is dropped with no reply on a connection that still answers, and a new thread asks again. **No approval request showed the content it approved**, in all eleven, so every *correct execution* above is judged against world drift, not against the instance semantics of §7.3. The countable cost is one look per approved dispatch; a first reading of wall clock was one tick of the harness's 25 ms poll and was withdrawn. *Bounds: scripted proposals whose recording is only partly pinned; one provider; the foreign writer is the test process; the external comparators are pre-registered and not yet measured.*

#### 7.5 A patch, and the traditional technique that beats it

OBSERVED E2E-1's patch workload never routed, so the question it was built for — a foreign edit to a part of the file the approved patch does not touch — was asked separately, with a wording

derived by rule from the capability’s own manifest and fixed before the run, so that it routes by construction. Three arms, two columns, every prediction held. **An irrelevant foreign edit makes the repaired runtime re-ask:** the binding is a whole-file digest, so any edit supersedes it, and the outcome is an unnecessary rejection by the gate. **The counterfactual column without the re-judge applies that patch,** and refuses the relevant edit anyway, through the provider’s own exact-match guard: *as measured* the compiler stamped every basis *advisory* and the provider checks freshness only when it is not, so *find* alone decided. The file’s own digests carry both halves: under the irrelevant edit the repaired runtime leaves the file as the foreign writer left it, while the ablation ends with the patch applied over it. **So for a find/replace patch, a content precondition the provider already has catches the drift this mechanism was built for, and the mechanism adds one re-ask and one look per approved patch.** That is a result about this effect class, and we report it as one rather than as a caveat. *Bounds: the ablation column is counterfactual — a real component under a mode this runtime does not produce, so it reports what the shipped system would do without the re-judge, not what any shipped system does; the wording routes by construction, so nothing here is a claim about how a model phrases a patch; one capability, one file, one foreign writer, a scripted model; find/replace only, since a patch whose edits depend on surrounding context is not covered; and the request never showed the patch, so correct execution is judged against world drift.*

## 8 Related work

**Agents acting on stale state is not new, and we do not claim it.** Recent work identifies *stale authorization* as a core failure mode of agentic systems — decisions taken from information available when an action is requested, committed as effects after budgets, inventory or approval status have moved — and defines a correctness condition requiring committed effects to be explainable against the policy state immediately before they occur [7]. Other work treats freshness as a first-class control variable, maintaining explicit temporal-validity ledgers alongside belief and arguing that validity is not recoverable from belief alone [29]. A related line reports that externalised procedural knowledge decays silently after a release, without raising any signal [6]. **And time-of-check-to-time-of-use in agents has its own dedicated study,** which introduces a benchmark for the class and combines prompt rewriting, state-integrity monitoring and tool fusing against it [27]; **we claim no priority over that result and do not contest it.**

**The closest prior work states our anomaly as a correctness condition and measures it.** *Commit-time authorization* [33] holds that a durable effect is authorized only if the witness that licensed its derived state remains fresh, causally prior, bound to the same effect, and eligible at commit time. It is evaluated on a controlled-invalidation suite over browser, tool/API and multi-agent workflows: in a 54-task matrix of 270 runs, endpoint success stays high while only 55 runs are authorized completions, and 207 of 216 invalidating rows commit after the authorizing path has failed. It ships CommitGuard, a fail-closed boundary monitor, with a validity-core ablation. **Our §4 anomaly is an instance of that condition, and we claim no priority over it.** We also record where that work is ahead of this one: its mechanism is ablated against its own validity core, and §7’s repair is not yet compared against the runtime it replaces.

**What is left, and it is not the word *observability*.** Against the policy-state and belief-ledger lines [7, 29] the distinction is clean: their staleness is *temporal*, a gap over state the governor can observe and close by ordering, while ours is a gap in what the observer could have seen at all — the freshness decision is derived from a counter authoritative for one provider’s own effects, so the runtime reports zero drift truthfully within its own observation model while the resource has

changed through a second provider, and serialisation does not close that gap because the change never enters the ordered history. **Against [33] that distinction does not survive as stated**, because boundary observability is structural there too, and its taxonomy of what incomplete observability hides is explicit. The difference is finer and it is the one this paper is about. That taxonomy is binary — a signal is emitted and the boundary can decide, or it is missing and the monitor fails closed — and its trust assumption is that the runtime, trace and monitor are trustworthy for the signals they emit. **Every failure we measure is a third state that binary does not have: the signal is emitted, well formed, and not load-bearing.** A declared premise is recorded, established by a look, and stamped at the one level that never blocks; a freshness verdict is emitted and it is proceed, computed correctly from a counter that cannot express a foreign write; the decided enforcement mechanism is reached zero times. A runtime in that state satisfies the deployment checklist a boundary monitor asks for — it exposes the witness, the dependency chain, the target and the eligibility path — while its own gates act on none of them. We report that as a finding about what such a checklist can certify, **not as a defeat of the monitor**: that monitor refreshes boundary evidence at commit rather than trusting the runtime’s verdict, and a file observation here records a content digest in its structured state — per entry, for a directory listing — so a monitor of that design would have something to recompute and would plausibly catch the case our own guard misses. **Which carrier holds the digest is itself one of our findings and we do not elide it**: the structured state carries it on every file read, while the *evidence* record carries it only on the branch that spills content to an artifact, and it is the evidence plane — not the structured one — that the completion assessment reads.

**Complete mediation is satisfied here, and that is the point.** The reference-monitor tradition asks that every access to every object be checked, and states *complete mediation* as a design principle in those terms [31]. **Every gate reported in this paper is reached, runs, and returns an answer**: mediation is not bypassed, and no result here is an escape from a monitor. The failure is one layer inside that principle — the monitor is consulted and its representation cannot express the change it would need to see. A principle stated as *check every access* does not by itself say *against what*, and this paper is a measurement of what happens when the answer to that second question is a quantity a foreign writer cannot move.

**The systems ancestry, presented as ancestry.** A basis is optimistic concurrency control [24]: act against a snapshot, validate it before committing. What differs is that a database validator sees every transaction in its own log, while ours sees one writer. The classical detectors for one resource with two writers are version vectors [1] and invalidation protocols [11]; a scalar counter is neither, and it is a degenerate case of the ordering primitive in [26]. Linearizability [20] names the correctness condition the policy-state work specialises; our anomaly is not a violation within a single history but the consequence of two providers over one resource not forming one history at all. Isolation-level analysis [10] supplies a method we do not yet have — name the anomaly, then say which level excludes it — and §4 weakens the need for it, since on the cross-provider path no level is consulted. Provenance semirings [19] are the formal ancestry for reconstructing where a fact came from rather than carrying it on the fact.

**Five shipping runtimes, read rather than measured.** To ask whether the representation gap is peculiar to this system, we read the approval and validation paths of five widely used agent runtimes. **None records, on an approval, any observation of the world that could later be re-checked.** Two compute a digest at approval time, and both cover the *call* rather than the world — one hashes the tool arguments, with a source comment saying it does so to tell similar calls apart. **This is a statement about what those systems represent, established by reading their source; it is not**

**a measurement of their behaviour**, and we draw no behavioural conclusion from it. Two further findings cut against us and are reported for that reason: rich approval vocabularies are common — one runtime’s human response is four-valued, another’s permission five-valued, and a third keeps granted and refused as separate fields, so *not asked* is distinguishable from *refused* — and one of them folds its five-valued permission into a single boolean in the durable record, which is the same collapse §5 reports here. **Neither the distinction nor the fold is peculiar to this runtime.**

**A marker that nothing enforces has an empirical literature, and it is months old.** §5’s second recurring shape — a plane that exists, is computed, and nothing reads — is not ours to name. Revocation enforcement in agent-memory systems has been measured across five systems, nine scenarios and nine models, finding that no system enforces revocation by default: the revoked fact is returned wherever the label is visible to the retrieval layer, outranks its replacement, and leads the agent to the unsafe action [34]. Making validity an explicit, revocable state of memory has likewise been built and evaluated [39]. A separate line shows memory granting authority the underlying history never permitted [12]; that is the opposite of our case, where the approval is genuine and correct under its own recorded contract and outlives the world that justified it. A framework audit makes the adjacent point that capability gating is not per-call authorization [28] — a question of *scope*, where the gate never asked about the argument, rather than of a gate that asked a question its representation could not form.

**A check that passes for the wrong reason has a name, and it is old.** In temporal model checking, *vacuity* is a property satisfied for a reason unrelated to the engineer’s intent, classically because an implication’s antecedent is never satisfied [25]. Our §3 finding is an instance, relocated from a specification to a runtime guard; we report the instance and our two cheap instruments, not the phenomenon. The empirical testing literature supplies the weaker neighbours: coverage correlating poorly with suite effectiveness [22], mutants as a substitute for real faults [23] — the warrant for the batteries described in Appendix A — and the oracle problem [9].

**On choosing a trigger, the literature is ahead of us in two directions.** When version comparison cannot fire, the trigger classes in use are lease expiry [18], writer-initiated invalidation [21], and read-triggered repair and background anti-entropy [14]. And the problem of choosing the interval was formalised in real-time databases, where each datum carries an absolute validity interval and update periods are derived from it as a scheduling problem [37], with later work on triggered rather than periodic updates [2] and on measuring staleness empirically under real workloads [8]. The security literature measures the same window from the other side, and widens it deliberately [3]. What does not transfer is the cost assumption: all of these treat a check as a message, where here it is an observation charged to a budget. The economics of deciding *which* checks to run has its own literature [30], as does controlling the overhead of monitors that run in production [4].

**Formal mathematics, as a comparison system.** §10 draws a contrast with a large collaborative formalisation whose interactive theorem prover enumerates every dependent when a recorded definition changes, so the failure reported here cannot occur silently there; where that project also lacks a mechanism — aligning a formal statement with human intent — the same shape reappears, and their own example of it is *vacuity* [13]. Separately, that literature uses *premise* in a different sense: an existing lemma retrieved from a library, with premise selection as a retrieval problem [38]. Where this paper draws a structural analogy between a justification and the effect it licenses, it is at the level of *propositions as types* only; the full correspondence requires an isomorphism preserving proof simplification and program evaluation [35], which we neither claim nor have.

## References

- [1] Comments on “detection of mutual inconsistency in distributed systems”. *IEEE Transactions on Software Engineering*, 1987. Cited for the version-vector construction of Parker et al., 1983.
- [2] Triggered updates for temporal consistency in real-time databases. *Real-Time Systems*, 2000.
- [3] Exploiting Unix file-system races via algorithmic complexity attacks. In *Proc. 30th IEEE Symposium on Security and Privacy*, 2009.
- [4] Adaptive runtime verification. In *Runtime Verification*, LNCS, 2013.
- [5] CAVA: Canonical action verification and attestation for runtime governance of agentic AI systems, 2026. arXiv:2607.13716v1.
- [6] Repo2Skill-Evo: Repository skills go stale in silence, 2026. arXiv:2608.21964v1.
- [7] Stateful governance for concurrent agentic systems, 2026. arXiv:2608.02764v2.
- [8] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. Probabilistically bounded staleness for practical partial quorums. Technical report, 2012.
- [9] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing. In *Proc. 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017.
- [10] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. A critique of ANSI SQL isolation levels. *ACM SIGMOD Record*, 1995.
- [11] Lucien M. Censier and Paul Feautrier. A new solution to coherence problems in multicache systems. *IEEE Transactions on Computers*, 1978.
- [12] Tommaso Cerruti, Mika Okamoto, and Ansel Kaplan Erol. Agent memory is a surface for endogenous authorization laundering, 2026. arXiv:2609.01836v1.
- [13] Johan Commelin and Adam Topaz. Abstraction boundaries and spec driven development in pure mathematics, 2023. arXiv:2309.14870v1.
- [14] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles*, 2007.
- [15] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [16] R. S. Fabry. Capability-based addressing. *Communications of the ACM*, 17(7):403–412, 1974.
- [17] Virgil D. Gligor. Review and revocation of access privileges distributed through capabilities. *IEEE Transactions on Software Engineering*, SE-5(6):575–586, 1979.
- [18] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proc. 12th ACM Symposium on Operating Systems Principles*, 1989.

- [19] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proc. 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2007.
- [20] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 1990.
- [21] John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, and Michael J. West. Scale and performance in a distributed file system. *ACM Transactions on Computer Systems*, 1988.
- [22] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proc. 36th International Conference on Software Engineering*, 2014.
- [23] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014.
- [24] H. T. Kung and John T. Robinson. On optimistic methods for concurrency control. In *Fifth International Conference on Very Large Data Bases*, 1979.
- [25] Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *Formal Methods in System Design*, 2001.
- [26] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 1978. Collected edition.
- [27] Derek Lilienthal and Sanghyun Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents, 2025. arXiv:2508.17155v1.
- [28] David Mellafe Zuvic. Capability gates are not authorization: Confused-deputy failures in LLM agent frameworks, 2026. arXiv:2606.28679v1.
- [29] Muhammet Ali Öztürk and Harun Artuner. ReSSERAct: Safeguarding LLM agent decisions against stale and contradictory evidence via typed hybrid state, 2026.
- [30] Gregg Rothermel and Mary Jean Harrold. Analyzing regression test selection techniques. *IEEE Transactions on Software Engineering*, 1996.
- [31] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [32] Igor Santos-Grueiro. Lingering authority: Revocable resource-and-effect capabilities for coding agents, 2026. arXiv:2606.22504v1.
- [33] Igor Santos-Grueiro. Temporary authority, permanent effects: Commit-time authorization for LLM agents, 2026. arXiv:2607.10487v1.
- [34] Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems, 2026. arXiv:2609.08258v1.
- [35] Philip Wadler. Propositions as types. *Communications of the ACM*, 2015.
- [36] Xiaoqi Weng. What you approve is what executes: Consent integrity for black-box LLM agents, 2026. arXiv:2606.02668v1.
- [37] Ming Xiong and Krithi Ramamritham. Deriving deadlines and periods for real-time update transactions. In *Proc. 20th IEEE Real-Time Systems Symposium*, 1999.

- [38] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2306.15626v2.
- [39] Yan Zhou, Yue Ouyang, Kaiyang Zheng, and Suncheng Xiang. TEPA: Revoking stale memories for conflict-robust language agents, 2026. arXiv:2608.07429v2.

## 9 Limitations

One runtime, two providers, one filesystem root, one machine, a single writer, scripted intents. Two results are counterfactuals about a guard rather than reports of runtime behaviour, and are marked as such where they appear. RQ5 was first measured at an in-memory run boundary and subsequently confirmed on the file-backed persistence path, so that caveat is discharged; it has not been measured under concurrency, multiple principals, or a model driving the runtime. Harness arms used a scripted look port that covers the targets it is asked about. In the shipped composition the binding look is formulated by the model, and an unusable reply falls back to a root listing of depth one, so a nested target the model does not name goes unbound and its approval’s one use is unchecked (measured on the daemon, E2E-1 W3). And an approval covers capability and resources, not content: §7.3 measures a re-proposal the person was never shown running on it. Two quantities are discriminating and not measurable here for want of a workload: the fraction of declared actions dispatching at non-zero drift, and the interval between the look establishing a premise and the act depending on it.

**No frequency in this paper is measured, and the reason is structural rather than an omission we intend to repair cheaply.** This runtime writes its journal into a directory created per test and removed when that test disposes, so no execution history survives to be counted. The arm ledger is not a substitute: every arm in it was chosen to separate two hypotheses, so a ratio computed over arms measures what we judged worth testing and not what a runtime does. Where a frequency would change a conclusion we say so and leave it open — one such case is whether a confirmation mode is worth building at all, which turns on how often a real workload takes the branch that yields a comparable identity. **A mechanism that is declared, wired, and never entered is a failure this study has measured four times**, so that question is not rhetorical; it is simply not answered here.

**Two limits on the strongest results in particular.** The three-way contrast of §4 compares three durable records *in one runtime*; the property it isolates — that carrying enough provenance does not imply anything reads it — is a claim about these three records, not about durable records in general.

**The second limit has since been partly discharged, and the part that remains has moved.** An earlier version of this section said the repaired runtime had not been compared against the runtime it replaces. That comparison now exists (§7), so the claim about *false-invalidations* is measured rather than withheld. **The claim about cost is not:** no latency, provider-call, observation-count or model-call figure appears anywhere in this paper, and the only statement about price is a worst-case bound. A mechanism that spends an observation per addressable resource is not shown to be affordable by showing it is correct.

**And the comparison introduced a limit of its own, which we state beside the result rather than after it.** Bound against a rule that keys on the lifetime of the episode that justified a grant — the validity predicate of a published revocable-capability reference monitor [32], reduced to that predicate and evaluated here on our own scenarios rather than run as a system — the shipped mechanism is sound exactly where that rule is unsafe — and unsafe exactly where it is sound: after the justifying episode closes with content unchanged, the shipped rule permits reuse in every measured cell, because content is unchanged and content is all it consults. **Content binding and**

**lifecycle binding are orthogonal, and neither subsumes the other.** No sentence in this paper reports the first half without the second.

**On generality.** §8 reports a source reading of five shipping agent runtimes in which none records, on an approval, anything about the world that could later be re-checked. That is a claim about *what those runtimes represent*, established by reading them. **It is not a claim about what they do:** no measurement in this paper was taken on any runtime but ours, and the architectural form of this result remains a hypothesis. **We make no claim that any of these findings generalise to agent runtimes at large.**

## 10 Discussion

A guard’s correctness and a guard’s reachability are independent properties, and testing establishes only the first. Scheduling discipline can close a correctness window and remove a mechanism’s purpose at once, with neither decision able to see the other. And a decision record can be correct in its reasoning and inert in its effect, for the same structural reason as the code it governs.

**Local correctness does not compose into a valid dispatch.** Figure 1 is three gates each returning a true answer and one dispatch that should not have happened. The bounded reading this study supports is that **local correctness of each gate does not imply validity of the composed dispatch when the gates do not share a representation of the world facts on which authorisation depends.** Here they do not: the precondition gate holds session and revision, the staleness guard holds a provider-local counter, and the authority record holds capability, resources and a timestamp — three vocabularies, no term in common that a foreign write moves. Each gate answers its own question completely, and no gate is in a position to ask the question of §4, because none of them holds a representation in which that question can be posed.

Read as interpretation rather than as a model, the validity of an effect appears to depend on at least four things being true together: that the decision’s reasoning was correct when it was made; that the world evidence bearing on it was observable at that time; that the authority record stayed bound to that evidence; and that a later execution can determine whether the binding still rules. This study measures failures of the third and fourth on one runtime, and the two are separable: the third durable decision of §4 *stays* bound to its evidence, with the provenance intact, and fails only the fourth — nothing on the path that reuses it attempts the determination. **That is the cleanest case we have that the last clause is not implied by the one before it. We do not offer this as a theorem, a formal model, or a necessary-and-sufficient condition,** and we have not shown the list is complete or minimal; establishing any of that is future work and would need more than one substrate. Its use here is to say which of the four each measured failure belongs to, which is a distinction the verdicts alone do not carry.

A contrast is available from a large collaborative formalisation whose theorem prover enumerates every dependent when a recorded definition changes: the failure reported here cannot occur silently there. Where that project also lacks a mechanism — aligning a formal statement with human intent — the same shape reappears, and their own example of it is vacuity.

### A Measurement discipline and claim revisions

This appendix records how the claims in §4 were obtained and, in four cases, narrowed after they were first written. It is kept out of the technical narrative deliberately: how we corrected ourselves should not be more prominent than what the runtime does.



## A.1 Pre-registration, and a distinction the ledger did not carry

Each experiment writes numbered predictions before measuring, and the outcome is written with the predictions left standing. Twenty-seven of the 272 arms carry a prediction that did not hold — eighteen predictions, because one refuted cost prediction spans ten cells and three E2E-1 arms share one cause. A further eleven were *not predicted at all*, two of them found by a surviving mutation rather than by anyone asking. Instrument predictions, about what a control or baseline should read, are recorded separately and are not in these counts.

**The two were counted as one for most of this study.** A single ledger field — an arm’s measurement did not match its expectation — was used for both *we predicted this and were wrong* and *nobody predicted this*. Fourteen arms carried it at the time, and every figure and count generated from the ledger therefore reported fourteen wrong predictions, **twice the true number**. The fix is two fields rather than one: an arm nobody predicted now records that no prediction existed and carries no match verdict at all, because without a prediction there is nothing for a measurement to match. We report this in place of quietly re-cutting the numbers, because the inflated figure was published in an earlier draft of this paper and the inflation ran in the direction that flatters the study — a discipline that catches more mistakes looks better than one that catches fewer, and *unforeseen* was being counted as *caught*.

**Three kinds of wrong prediction, and how the figure came to show them.** By prediction, the eighteen divide by *who* was wrong. Fourteen were wrong in a pre-registration, one of them the cost measurement’s, which spans ten arms; among the others, one predicted that a compiled action would carry a particular freshness mode, and the action carried no basis at all — relocating the finding one layer earlier than the prediction had reached. Another was *right for the wrong reason*: a census was predicted to find one capability producing facts without identity, and found two, the second being the directory case an earlier experiment had already explained. Recorded as confirmed it would have hidden the more important fact, that directories are the general case and deletion the special one; a prediction that needs re-reading to look confirmed was not confirmed. The third named the wrong *cause* for a verdict it called correctly, predicting that a move’s absent destination would fail to bind because the lookup failed, where in fact nothing had asked about it. A fourth expected a repair to reach a case it does not reach (§7). Three more were wrong in a design document rather than in a pre-registration. **The last was held by the implementation and written down nowhere.**

That three-way attribution was for a time *not* a field the ledger carried, and an earlier version of Figure 4 derived it by pattern-matching the prose of the expectation field — which is how one arm came to be classified from a sentence that contradicted the experiment’s own plan file. The figure was cut back to what the data actually distinguished, the ledger then gained a field for it, and the attribution is now read from that field rather than inferred. We record the sequence because the intermediate state was the honest one: **a figure should lose information it cannot source before it gains a field, not after.**

**A refuted prediction that later came true, and why the wording was never touched.** One pre-registration expected a move’s absent destination to fail to bind because *the lookup failed*; it was measured as *nothing ever asked*, and recorded as a miss with its wording left as written. After looks were made to cover what a binding needs, the destination is now actually looked for, the look returns a not-found error, and the recorded reason is the one the prediction named. **The prediction was right about the world and wrong about the runtime, and it is true now because the runtime changed rather than because anyone re-read it.** Had the arm been amended to its measured value when it missed — the tempting tidy-up — this would be invisible today: the ledger would

show a prediction that always held, and the fact that a repair moved the runtime onto it would have no trace. This is the clearest return we have on retaining refuted predictions verbatim.

**A pin that moved for a reason nobody predicted.** A later pre-registration committed in advance that if a particular revalidation pin moved, that would be a finding rather than a fixture to update. The pin moved. **But the stated mechanism did not occur:** the prediction attributed the move to extra facts overturning a verdict an earlier look had settled, and what actually happened is that the resource in question was unsettled precisely *because nothing had ever covered it*, so the second look settled it for the first time. No settled verdict was overturned. The arm was kept for its purpose — it is the only shape in which two particular states co-occur — and rewritten to state the premise it had been getting for free. **The prediction named the right pin and the wrong mechanism**, and recording that distinction is the reason the commitment was made in advance.

**A field that asserted an event that did not occur.** Two arms of the daemon grid were named for the grid cell at which a foreign write was to be injected, and the field recording that cell was defined as *where the foreign write lands*. In those two arms nothing landed: the intent never routed, so the hook that fires between answer and dispatch was never reached, and their evidence records no post-injection digest and a final world equal to the seed, in both columns. Every outcome field was correct — both arms are recorded as *not reached* — and no table counted the field, so nothing downstream was wrong. **What was wrong is that a present value asserted an event that had not happened**, and it survived because the arms' names, their pre-registered cells and the field all agreed with each other and disagreed only with the world. It was found by re-deriving the digests from the recorded file contents rather than by comparing two derived digests, and the fix separates the two questions one field had been answering: where an injection landed, and where one was specified. **A field records what was measured; what was intended gets its own field rather than borrowing the measured one's vocabulary.**

## A.2 The defect repaired, on the same daemon

OBSERVED §7.3's failure was ruled a blocker and repaired, and the repair was measured before it was described. The approval's identity now includes the proposal's canonical arguments — a key the runtime already computed and already pinned, whose gate previously excluded every effect that named a resource, which was precisely the defect's surface. Three single sites, and the same three arms through the shipped binary.

**The cell that executed an unapproved proposal is now a correct rejection**, and world truth carries it: the second write is never on disk, counted off the target's bytes, and the dispatch record names no authorising decision. The arm that re-proposes identically still dispatches, on the person's own decision, with every cost field unmoved. **The cost is one extra approval, charged to that cell alone** — the third arm already asked twice before the repair. Reported as three arms, not four: the fourth prediction concerned a decision recorded before the field existed, and this harness cannot produce one, because the client answers and the *daemon* builds the decision from the escalated effect; that prediction is evidenced at gate level instead.

**Two consequences are reported because they are true, not because they were wanted.** The repair invalidates *every* approval recorded before it, including the legacy ones an earlier repair was deliberately careful not to invalidate — a mechanism that fails closed on every stored decision is a different proposition from one that fails closed on drifted ones, and a reader has to be told which. And the proposal's content now reaches the request twice over, as the question id and as a parameter, where eleven earlier requests carried none. **That is disclosure, not presentation:** a canonicalised argument blob in a field is not what a person would need to read, so **it does not discharge the requirement that a request show the content it asks about**. Both are recorded

in the ledger, which now distinguishes the two columns by a field for whether the request carried content.

**And the repair reaches a defect its own scope did not name.** An earlier measurement had an approval to run one command dispatch a different one, because both named a resource and the coarse key applied; the repair of that era declined the case explicitly. Instance identity closes it, the arguments differing, and the arm was built to escalate *for that reason* rather than because a legacy decision matches nothing, which is what makes the result readable at all. *Bounds: a scripted re-proposal, one capability, one resource; how often a model re-proposes differently is unmeasured; and the comparison against the unrepaired runtime is against its recorded rows under this harness rather than a re-run of it, which is weaker than a column and is weaker for a reason this paper documents — the outcome of this suite is not reproducible from a commit alone (§B).*

### A.3 Two shipping frameworks, run

OBSERVED The predicate comparison above reduces published designs to validity rules. This runs two frameworks instead, unmodified from PyPI at pinned versions whose wheel digests match the index's published ones, in six configurations a developer could ship: LangGraph's `interrupt/Command` resume with the plan taken before the pause and with the node re-executed, and the OpenAI Agents SDK's approval under its default fingerprint, under a changed call, under a standing grant, and behind a developer-written `needs_approval` callable. Sixteen arms, one file, one foreign writer, a scripted model. **Every arm executed, and no framework declined on a changed world.**

**Among these frameworks, on relevant drift the mechanism is alone in refusing, and on irrelevant drift it is alone in re-asking.** Both belong in one sentence: the refusal no comparator here made and the unnecessary re-ask no comparator here made are the same design choice seen from two sides.

**The unqualified sentence does not survive whole runtimes, and a later column measured that.** Two closed-source agent runtimes were driven through the pre-tool hook each exposes — the programmatic seat of the approval decision — with a real model, eight cells, controls included. **Both refuse a proposal formed against the pre-mutation world on their native editing path:** one will not save a file that changed after its read, the other's patch fails on a context line that no longer matches. **Neither refusal is approval revalidation; both are content preconditions,** the class of §7.5. **The same runtimes, the same approval semantics, one command rewriting the whole file: both execute the stale proposal.** So the discriminating boundary is the effect shape rather than the runtime, and it runs through three independent implementations at the same place. *That column's own bound is sharper than its result: its hook answers every call, so neither runtime reuses an approval, and it is therefore silent on the property this paper is about.*

**What reconciles this with §7.5.** Those two results look contradictory — a traditional technique beat the mechanism on patches, and here nothing else refuses — and they are not, for a reason that is about effect shapes rather than about frameworks. A find/replace patch *carries its own content precondition:* its `find` string stops matching once the text drifts, so the provider refuses without any approval being re-checked. A whole-file write carries none, and *as these arms were measured* the provider's freshness check did not supply one — the compiler stamped every basis *advisory*, and an absent basis means *freshness does not apply* rather than *assume fresh* (both read from source). **So the mechanism's unique contribution is on effect shapes whose content is not already a precondition; where the content is one, it duplicates a cheaper guard and adds a re-ask.** That is a narrower claim than either result alone suggests, and it is the one the two together support.

**The advisory literal was later removed, and the grid did not move.** The runtime already held a whole-file digest precondition, supplied by the runtime rather than named by the proposer: the

provider implemented the stricter modes and compared the current digest against the basis observation. **A literal in the compiler switched it off**, overriding the schema's own safer default at every call site at once, under a written rationale that a path-addressed write cannot go stale because its target resolves by name — true about the target and silent about whether the bytes being replaced are the bytes anyone read. **That is why the gap survived being measured, recorded, and built upon: a gap with no stated reason gets revisited, and a gap whose reason answers a neighbouring question does not.**

When the literal became a parameter and the whole-file write gained the enforcement the patch already had, **no cell moved in either column**. At that commit the reason was parasitism: an action carries a basis only when the run that compiled it took a look *itself*, and in the ablation column the dispatching run never looked — its events were a goal, a proposal and a dispatch, and every committed dump of that column up to then reads zero looks for that run. No look, no basis, nothing for the new precondition to compare. Under the mechanism the same run looks twice, both times for the re-judge and the binding — a figure first recorded as one, and corrected here against the committed dumps. *We bounded that result to the implementation: a runtime whose dispatching run observed for its own reasons would hold a basis with no approval machinery.*

**The runtime then became that counter-example, and the separation moved.** A later decision made the runtime look before it changes something, for its own reasons: once authority decides to dispatch a non-read effect whose resources the run has not observed, it takes a covering look and regrounds the action. Both columns were re-run against predictions committed beforehand, and all six held. **The ablation column still overwrites, for a new reason:** its dispatching run now looks — the stub prompt names the look as the pre-change look — but after the foreign write, so its basis is the changed file's digest, the provider finds nothing moved, and the write lands. The approval had recorded the seed digest when the person answered, and under the mechanism the gate compares against that and refuses; none of that column's looks is a pre-change look. **A witness taken at dispatch, however it is caused, is taken after the decision and cannot see what changed in between; only the witness recorded at the decision can.** That is the call-versus-decision distinction, measured on an existing cell, and it no longer depends on whether the runtime looks. On the unchanged-world control both columns spend two looks, so the mechanism's marginal look cost against a runtime that already grounds its changes is zero. **That shared look had a cost the count hides:** the re-judge's covering look happens after compilation and satisfies the pre-change look's coverage test without grounding the action, so in the committed journals of both commits every approved top-level dispatch under the mechanism went out with no basis, while every dispatch in the ablation column carried one. Adding the decision-layer guard switched the execution-layer guard off for approved writes. A repair grounds such an action on the look that covered all of its resources; re-run on the grid, every dispatch under the mechanism then carries a basis from its own run's look, and no gate or final world moved in either column.

**An unattended mission measures the same window where no decision was recorded.** With authority delegated and no client attached, a mission first *never looked* — the runtime observed only to resolve an unknown or to bind an approval before asking, and a delegated mission needs neither — so it overwrote a foreign write, verified from its own receipt, and recorded itself completed; with the write withheld it blocked and the foreign write survived. Three decisions later it recovers without a person: the pre-change look put a basis on the write, a write landing after that look was refused, and once a refusal stopped being credited as success when a look showed the assertions holding, the mission was diagnosed, re-observed, applied, verified and completed with its own content. **A write landing before the pre-change look is still overwritten:** the look grounds the action on the world it finds, not the proposal on the world the proposer imagined, and on this path

nothing recorded what the world was. *Bound: scripted transport, one desired effect, one foreign writer, runs of a fraction of a second.*

**A real model in the mission loop, and what it does not reach.** Seven small mission tasks with a real model proposing and a checker that imports nothing from the runtime, one run each. The checker passed three of six at the first commit, when the proposer was blind to what the runtime had observed and re-asked for reads until a repeat guard stopped it; five of seven once it was shown what the runtime saw; seven of seven after three further decisions, with the runtime declaring completion on six. In twenty rows across the three commits no row declared complete was failed by the checker. On the same tasks Claude Code passed seven and Codex six, Codex's failure being an undo of a foreign change it had seen, inferred from the reads around an edit whose diff its event stream does not record, under an objective ambiguous about *as it is* when. **The suite does not separate the runtimes, and no run in it requested an approval**, so it says nothing about the mechanism; an approval task built to reach it then ran with every row valid and **no row reached revalidation**: the next run re-proposed the approved write with different optional arguments in five of seven rows, so instance binding asked the person again before any world comparison — in both unchanged-world rows under the mechanism, unnecessarily. In the re-judge-removed column one matching re-proposal met a foreign write on its own read and was refused by the provider's basis check alone. With the approved proposal re-submitted and the approved write grounded on its covering look, the re-run's seven predictions all held: a change after the answer was refused by the gate under the mechanism and written over without it, where the ablation's basis digest equals the changed file's; a change after the run's last look was refused by the provider's basis in both columns; unchanged-world rows executed with no second question. One run per cell.

**Presentation, on the mission path.** A terminal interface to missions presents each question with the proposal's full arguments and the identity each resource is bound to. One hand-run session with a real model was asked, shown the patch and the file's digest, approved, and completed, with the approved dispatch grounded on a look of its own run. An earlier attempt whose model transport had lost its login applied the approved patch and then could not evaluate the criterion, because the only observation covered the file's state without its content and every model call that could have asked for more failed: an approved write can act when the proposer cannot. The runtime track recorded that attempt as invalid, and it is stated here as a design bound only. The daemon's wire requests still do not show content.

**A framework executes a proposal its user never saw.**

**A framework executes a proposal its user never saw.** LangGraph's re-executing configuration is *right about staleness* — the node re-observes the world, so the plan that runs is always fresh — and matches the resumed approval *by position*. Approval was shown `retries = 3 -> 5`; the world drifted to 4; the plan that ran was `retries = 4 -> 5`, and it ran. **This is the seventh outcome bin, instantiated in software we did not write**, and it is the same class as §7.3: an approval that covers a question rather than the proposal a person was shown. **Instance binding is therefore not an artefact of our implementation.** Right about staleness, wrong about consent, and the two must not be collapsed.

**The developer-written check is asked once.** The SDK invites a `needs_approval` callable, which is the obvious answer to *a developer can simply add a check*. A digest-comparing callable was written and **evaluated exactly once, at the first turn, in all three arms**, and never re-evaluated on resume. So the witness fails not because it is too weak but because nothing asks it again — which

is this paper's question in executable form, answered against one version. The corollary is that the over-rejection of §7.5 cannot arise there either: a check never re-asked cannot re-ask wrongly.

**Each comparator is right about something.** The changed-call configuration **re-pauses** when the re-emitted call carries different arguments: real revalidation of call identity, and the only refusal anywhere in the column. The re-executing configuration never runs a stale plan. The prior-plan configuration is right whenever the world does not change, and is the cheapest of the six. *Bounds: these are frameworks, not runtimes, and a developer can add checks — the finding is that the invited check is not re-asked, not that it cannot be written; the model is scripted, so differences in re-planning are unmeasured; there is no journal, so every observable is the harness's own, read from the file's bytes; two frameworks at two versions, one file, one foreign writer, one machine. Every effect applied in this column is a whole-file write — `path.write_text` in both LangGraph cells, a `write_file` tool in the SDK cells, and `fs.write` in our own arm (all read from source) — so what a patch describes here is only what the person is shown, never what is applied. That is why the two results are commensurable: this column probes the shape where the mechanism is unique, and §7.5 probes the shape where it is duplicative. The comparators were not given a find/replace tool, and that configuration — where a content precondition would do this work without any framework — is unmeasured.*

#### A.4 Mutation batteries, and what a survivor means

Every experiment ships a battery. A surviving mutation is answered as one of *untested branch*, *weak assertion*, or *behaviourally identical for the tests given*. The three are not interchangeable, and the third is the most easily misread: in one case a guard survived not because it is unreachable — it is live production code with two call sites — but because the test list contained no case driving it. **A battery is only ever meaningful for the tests it is given.**

Twelve instrument gaps were found on this line; five were caught by a battery. **The count moved from ten and the two additions share a family the earlier ten did not have**, described in the next paragraph; it is restated rather than incremented because a hand-maintained count on this line has drifted before. The distribution of the rest is the more informative number: an assertion that could not discriminate between a correct implementation and a broken one; a field-selection error where an assertion was satisfied by the one field a mutation had not broken; a fixture that deadlocked and would have reported zero activity as a result; a mis-targeted mutation aimed at a branch its input could not reach; and one of a different kind — *sample selection*. That last filtered a measurement to a single file and asserted a property that held. It was true, and it measured the one case that could not exhibit the effect, which appears only for directories. Had it stood, the paper would have reported the weaker conclusion that the experiment's own pre-registration was written to prevent.

**A check whose failure mode is silence, which is the family we had already named.** Two comparators central to the comparison of §7 compared absent values. Each read a cell out of a keyed collection and tested the two sides for inequality, so a pair of missing lookups compared *nothing to nothing*, found them equal, and reported **agreement on a comparison that never happened**. A grid that had shrunk, or an arm that had been renamed, would have produced a *passing* control. **It matters most for the one experiment whose finding is a null:** an empty loop returns exactly the shape a null result has, so the result that must never be reachable without measuring was reachable without measuring. Both now fail loudly, naming the missing cell, with a case asserting that the guards change no result on the real grids so the repair stays an instrument correction rather than a measurement change.

**A third instance was in the toolchain that renders this paper.** The typesetter was absent from the machine, so a scripted rebuild did nothing and every subsequent check read a log written the previous day; a query for unresolved references returned zero because it was reading a stale file, and the paper's page count was misreported for an entire working session.

**What makes these worth a paragraph is not the bugs but that this codebase had already named the family.** A reproducibility checker in the tree documents, at length, that it would pass by resolving nothing — and three more instances shipped anyway. **Naming a failure family does not immunise against it**, which is the same conclusion this appendix reaches about the two-planes family, from independent evidence. **The rule that follows is cheap and neither track was applying it: before trusting a green comparator, assert what it returns on empty and on truncated input.** None of the three was caught by the check itself; each was caught by someone asking what the check would do if it were broken.

**What the pre-registrations bought, measured by what actually found things.** The old-vs-repaired comparison of §7 grew three times while it was being run, and **not one of the three additions was found by a test going red.** Each came from someone reading a written prediction against a produced result. The first: the set of reuses labelled *ought to stand* contained no scenario in which anything had changed, so a mechanism that escalated on any observed drift would have passed all of them, having seen none. The second: the arms that answered the foreign-write row went through the runtime's own coordinator, which moves the counter, so they measured a self-caused change and the row named a foreign one — neighbouring objects sharing a word. The third: every arm at a resume boundary differed from a same-world control by a write somewhere, so the one row the block calls prominent had no cell.

Two of the three closed a gap in what the set could discriminate; the third closed a control. What they share is not their kind but their instrument. **A pre-registration's value here was not that it stopped anyone reporting a result they had not predicted — it is that a written prediction is a thing a second reader can hold against a measurement and find insufficient.** A green suite cannot do this, and neither can a battery: both answer whether the arms that exist agree with the code, and the defect each time was in which arms existed. We report this because the discipline is usually justified by the errors it prevents, and on this line its larger return was the arms it caused to be added.

**One refinement, from the cost measurement, and it separates two kinds of pre-registration.** That experiment's plan named a control written months earlier: *the same path with revalidation disabled*. **It was the wrong control** — that anchor removes the mechanism which *consumes* an identity, while the observation is bought earlier, so the cost stays in place and the mechanism would have measured as free. It was caught because the plan also carried a prediction about *the baseline itself*, which failed. **A pre-registration that predicts only outcomes would not have caught it:** the outcome prediction was about what the mechanism costs, and the error was in the instrument that was to measure it. Predicting something about the instrument — what the control should read, what a comparator returns on empty — costs a sentence, and on this line it caught what no outcome prediction could.

**Four ways one checker reported success.** The reproducibility column of the results ledger names, for each arm, a commit a reader can check out. It drifted silently on three experiments before anything checked it, so both tracks added a check — and the checks then failed in four distinct ways, all found within a single sitting:

| what the checker did                  | what it reported                                         |
|---------------------------------------|----------------------------------------------------------|
| resolved the wrong file               | failures that were not there                             |
| resolved nothing                      | success (a relative path, read from the wrong directory) |
| could not resolve, by design          | success, more quietly                                    |
| resolved a file that was always there | success, <i>while looking as though it had checked</i>   |

**Only the last survives both a green suite and a deliberately corrupted input**, which is why finding it required corrupting one on purpose rather than observing the check pass. Its cause was that the question being asked — does the recorded commit *contain* a test file for this experiment — is satisfied by construction whenever an experiment’s arms are new assertions inside older files, as two of these seventeen are: every commit since those files were created contains them. The rule is now that the commit must have *touched* one of the experiment’s files, which is the question the column exists to answer. The generator that prints the column applies it — the other prints no anchors and carries a different rule — and we verified it the way the fourth row demands: green on the correct ledger, red on a deliberately corrupted one, naming the experiment and the files its recorded commit did not touch. Adopting it surfaced two more instances of the second row in one script of ours — a relative input path and a relative output path, the latter having quietly written a second copy of a generated table elsewhere while the paper kept a stale one.

**The third and fourth rows are the same failure this paper reports in the system it studies**, occurring in the instruments built to study it: *nothing could be resolved* read as *nothing to check*, which is absence taken for assent. **We deliberately do not count it among the instances in §5**, and the reason is the one that makes that section’s claim checkable: its refutation condition quantifies over planes of the runtime under study, and an instance drawn from our own tooling would leave a reader unable to tell what would refute it. The observation is real and is reported here, where instrument failures belong; it is not evidence about the runtime.

**A test that would have passed for the wrong reason.** The most recent experiment’s plan asserted that a capability manifest states its capability’s parameters, and concluded that standing up the real provider would be more machinery for the same answer. **The manifest carries a hash of the schema, not the schema** — a deliberate two-level design, its loader recording the measured motivation of 300 to 2000 tokens per tool schema. The first version of the test therefore read an empty parameter list for *every* capability, which is the shape the experiment was looking for: its census would have been confirmed on the finding that nothing names a location. It was not confirmed, because a rule adopted one experiment earlier requires each arm to assert *what it routed to* as well as what came back, and that assertion failed. The premise was ours and it was false; what caught it was a discipline adopted for an unrelated reason, and it caught it before the number existed rather than after it was published.

## A.5 How four claims were narrowed

The sharpest sentence in this study moved four times, each toward a less quotable and more accurate one: that the strictest mode has nothing to be strict about; that the mode is never consulted and the enforcement seam is dead; that the check fires only after something unrelated happened, so coverage belongs to the workload; and that three causes share one verdict.

A fifth and sixth correction arrived together and are worth reporting as a pair, because they share a shape and run in opposite directions. Both generalised a locally true fact to a larger claim without re-checking the larger one: reading a decision record’s statement that it authorises no spending of its own as a refusal of future authorisation, which made an open question look harder than it is; and generalising *the binding is bought once to the mechanism costs one observation*, which made it look cheaper. **The second is the more dangerous, because it favoured the direction we wanted:**



the first costs someone extra homework, the second would have had a person decide against an understated price. Neither reached a deliverable.

We traced how each correction started, because the tempting summary — that one method operated at several levels — is a narrative we could not support. Only the first came from an instrument, a mutation aimed at something else. The second came from review, the third from an incidental corollary while writing up an unrelated result, and the fourth from a confirmation question asked by the track consuming the result. The third refuted nothing about the system; it refuted the representativeness of a fixture, a failure that leaves every assertion green and every prediction confirmed. The claim we defend is therefore narrower than the sequence suggests: **this discipline reliably turns a suspicion into a checkable result, whatever its source; it does not reliably produce the suspicion.**

Two further corrections are recorded because they run against our own interest. We first wrote that the machine-readable ledger caught the fourth error; it did not, having been generated from the same wrong belief, and a confirmation question caught it. And the step that reframed *why is there no detector* into *why does the detector not cover approvals* — which prevented the wrong conclusion that a detector needed building — followed a standing instruction to check existing abstractions before proposing a parallel one, rather than originating with the track that carried it out.

#### A.6 A true sentence about the wrong object, three times

The error §3 reports — a correct statement carried to an adjacent object without being re-derived there — occurred three times on this line, and the third occurrence is the reason we give it a section rather than a clause.

The first was a claim about one pin's timing, true of the pin it was made about and false of the second pin it was applied to. The second was a description of a mutation guard, true of the call path it was read on and false of the two production call sites it was generalised to. The third was a sentence about a gate's comparison — *the check is not a revision comparison* — true of the branch that fires when a session is gone, and false of the gate, which compares revisions on the next line. **It was written one paragraph below the passage describing this exact error**, in the same document and the same commit.

**How it gets in.** The third came with its own quotation. The source lines were quoted accurately and the sentence generalised from them, and **that combination is the mechanism**: the quotation looks like evidence for the sentence, so a reader who verifies the quotation sees agreement. What is unverified is not the quotation but its *extent*. What caught it here was reading the lines immediately outside the quoted range.

This belongs with the instrument failures above rather than apart from them. A checker that resolved a file which was always present reported success while appearing to have checked; a quotation that is accurate about too little does the same thing to a reader. **Verifying the citation is not verifying the claim**, and in both cases the failure is invisible precisely to the check a careful reader would run.

**A wrong conclusion that was never written down.** Every correction above followed one shape: something was asserted, and was later caught. One did not, and we report it separately rather than in that list, because a list of errors that were all eventually caught misrepresents what happened here.

Measuring the branch above began from a refuted prediction — a directory listing was expected to mint evidence and mints none. From that true result, the inference *then no criterion can be resolved by a listing, so recovery can only ever replace* follows, is wrong, and was one edit from

being recorded. It is wrong because it generalises from one branch of a three-branch function to the function. **No test caught this, and none would have:** every arm agreed with the reading, and the claim was about a branch no arm addressed.

What stopped it was unrelated to it, and less creditable than our first account. Two mutations in that slice's battery were *refused reporting* — one anchor matched nothing, one matched two sites — which forced a return to the provider source. That much the guard did. But the refused anchor concerned a mutation about minting evidence, so finding a replacement meant searching the provider for *evidence*, and it was that search term — the concept the unmeasured branch is about — that put the branch on screen. **Had the refused anchor concerned anything else, the same guard would have sent the same reader to the same file and surfaced nothing.** Two things were needed and only the first belongs to the instrument. We state it at that width because the wider version flatters the instrument, and **an instrument credited for luck is one nobody will improve.** The transferable part is narrower than the anecdote: **a claim of the form *X never happens*, derived from one branch of a three-branch function, is not a measurement of X** — and neither the pre-registration nor the arms exposed that, because both were written about the branch in hand.

**A result that passes is never read again.** Every family above is a check that fails quietly. This one is its opposite, and it cost more. **A passing result emits no signal, so nothing draws a second reader back to it.** The strongest finding of §A.3 — a framework we did not write executing a proposal its user never saw, which makes §7.3's failure a class rather than a defect of our implementation — was recorded as a cell that matched its prediction, because it did match. Its significance went unread until a second reader asked what the cell *was*, separately from whether it agreed. Nothing was wrong with the check, the prediction, or the record. **The practice that follows: when a slice finishes, re-read the arms that passed and ask what each demonstrates, separately from whether it matched.** The ones that failed a prediction get that reading for free; the ones that matched receive no other review.

**A verified cause is not an exhaustive cause.** The repair of §7.3 put the proposed content on the approval request, which no prediction had. Asked *how*, this line named one path, and a reader verified it at source and found it true: a field the request builder already copied. **It was true and it was not the only path.** The question id independently carries the same content, because the repair appends the occurrence to it and the request's item id *is* that question id. Removing either path left the content on the wire by the other, and two successive single-cause accounts each passed a check written for it. What settled it was not a better reading but an **exhaustiveness** assertion — naming both paths and requiring that nothing else in the request carry the content — which fails if a third appears. **A causal claim about an observation is not established by verifying the cause it names; it is established by excluding the others.** The cheap discipline: when a check confirms a mechanism, ask what else would produce the same observation, and make the check say *only this*.

**A second reader who inherits a summary is not a second reader.** Two errors on this line were the same sentence written from opposite directions: a framing one track had not re-derived, and that framing taken on trust by the other, who had no reason to doubt it. *Every cell produced a measurement* travelled from a report into an audit verdict while being refutable in seconds from the table directly beneath it — one cell had a single arm, which is one of the study's own non-measurement categories. A bound in the same document travelled the same way into two papers, and was false in the direction that *understated* the result, which is the direction a careful reader does not think to check. **Independence is a property of the derivation, not of the person:** it buys nothing where the second party re-uses the first party's sentence. The fix is this paper's own rule, turned inward — check the artifact, not the report of it.

**Checking, by a method that carries the error.** Both tracks verified the census above independently, both said in advance that a claim about consumers must be checked rather than repeated, and both then counted occurrences of a name with a text search. Neither search excluded comments. One census consequently listed four modules that mention the verdict only in prose as though they acted on it; the other listed five that do act on it and missed two more. **The two errors are not equivalent and we do not report them as a matching pair:** an over-count rests a claim on readers that do not exist, while an under-count leaves a true claim weaker than its evidence allows. What the exchange establishes is narrower and worth more than a symmetry — **the check was performed, and the method of checking was itself the carrier of the error.**

**Doing the same check more carefully did not repair it.** Told that comments were being counted, the over-counting track excluded them and counted again, and over-counted a second time by a different mechanism: the name also appears in message strings, and once in an unrelated sense entirely — a comment on audio URLs that happens to use the word *undecidable*. Three methods produced three numbers, and the one that was right was the only one that asked, of each occurrence, what that line was doing. The discrepancy was not found by reading more carefully; it was found by counting a different way. The rule this yields is not *verify rather than repeat*, which both tracks already held and both followed here. It is **that an independent check inherits nothing from its independence if both parties reach for the same instrument**, and that a count of names is not a count of consumers however carefully it is filtered. **What the independence did buy was a disagreement.** Two counts that differ force a third method; one figure arrived at twice forecloses it. Had the lower number been accepted, the record would hold a consistent count, wrong in the direction that understates, with nothing in it to prompt a recount — and no amount of re-reading produces a second number, which is what makes a second party worth having even when both are wrong.

One filter in this study acted earlier than any of these and cost nothing: a claim that would have needed a judgement neither party had checked was not carried into the paper at all, and was refuted when someone later checked it. **We report the mechanism and not a merit.** It was not foresight — the claim was not believed to be false, only unverified — and it catches exactly the claims someone notices they have not checked, which is a set with no known relation to the claims that are wrong. It produces no signal at all about the rest, and it is the only device here whose successes and failures are equally invisible.

**That yields a design rule for instruments, not only a caveat about this one.** A guard that refuses *audibly* can be audited: this study's mutation runner prints a refusal and exits non-zero when an anchor matches no site, so the next action cannot proceed without answering it, and the ledger's anchor check reports an experiment it cannot resolve as a failure rather than passing over it. A guard that declines *silently* cannot be audited at all, because a claim it stopped and a claim that was never made are indistinguishable afterwards. Both kinds are present in this work, and the anchor check is the clearer case: its first version passed silently over anything it could not resolve, and it was rewritten specifically to make that refusal loud — **a change from a filter that could not be evaluated to one that can be.**

**A citation that has outlived its tree.** There is a second way the same trap closes. A line reference is a claim about one tree, and the file moves; a reader who follows it later lands somewhere plausible and unrelated, and the agreement they see has never been checked by anyone. We found four such references in our own records, correct at the commit their result was measured on and wrong against the current head — and our first correction was to repoint them at the head, which replaces one true statement with another that will rot on the same schedule. **The rule we adopted instead is that a line cited as evidence names the commit it resolves in**, exactly as a measured arm does, while a line cited for navigation is not dressed up as evidence by giving it one. Existing references are

not backfilled: what supports a result is its arm and its battery, and a drifted line misleads a reader without weakening the finding it points at.

The two failures in this section share a form with a third that §3 reports and one in §7. A statement true of one branch, reused for the whole gate; a statement true of one pin, reused for another; a line true of one tree, read against a later one; and a description of what an approval records, left in the present tense after the repair that changed it; and a rule forbidding one *direction* of a substitution, cited against the opposite direction of the same substitution. **That last is the hardest of the five to see**, because the two objects are not merely adjacent but share their vocabulary entirely — the same pair of facts, the same two values — and the quoted half of the rule reads as though it settles the case. It was caught by reading the rest of the sentence that had been quoted. **In none of these did anyone write something false.** Each was a true sentence that outlived the thing it was true of, and none of them is caught by re-reading the sentence.

### A.7 Two ways to support a sentence that look identical

The claim above — that the reused record carries the provenance nobody reads — was first supported by reading the provider that mints evidence and confirming it populates all three fields. That is a fact about the minting path, and the sentence it supports is *the fields are populated*. The sentence the paper needed is different: *the record this run bound carries them*. The two are separated by every step between minting and binding, and an argument for the first is not an argument for the second.

The distinction was raised across the two tracks and closed by measurement rather than by reasoning about the intervening steps: the arm now reads the evidence out of the resolved unknown and asserts each field. **We record the intermediate state because it is the more common one** — a claim resting on a mechanism that would produce it, rather than on an observation of it — and because from the outside the two are indistinguishable in the finished sentence. A mutation was also added that strips the world revision at the minting site, so that a later change quietly returning this finding to a missing-field result turns an arm red.

**A battery that refused to report.** That mutation's first version named a site that matched nothing. The runner refused to report the run rather than counting the mutation as killed: had it scored, a mutation *never applied* would have been recorded as one the tests caught. This is the same family as the checker failures above — **a check that did not run reporting success is the quietest way an instrument fails** — and it is the one case in this study where the guard against it fired on its own.

### A.8 A control that made another arm readable

Controls are a third of the arms in this study, and their usual job is to show that a measured effect is caused by what the experiment varied. One from the most recent slice did a second job worth naming. The slice's headline arm shows a flag reporting *covered* for a request whose look then found nothing — but on its own that arm is satisfiable by an instrument that reads no facts at all. A second arm supplies the other direction: a request naming nothing, whose look *does* return the target. **Without it the first arm would pass whether or not the measurement worked**, and with it the pair also constitutes the finding, because the two together show the flag is not reading the result in either direction.

That is the shape a census one slice earlier confirmed its own hypothesis by, for the wrong reason, until a rule adopted after it required each arm to assert what it routed to as well as what came back. **A control is not decoration here; it is sometimes the only thing standing between a finding and an instrument that would agree with anything.**

## A.9 What was deliberately not measured

Two provider surfaces are unmeasured for identity coverage. The reason is a standing rule that a measurement needs a reader rather than merely being runnable: those surfaces have no consumer today, so measuring them would characterise a plane nobody uses. They are recorded as unmeasured rather than filled in, and the blank cells in Figure 4 are that decision made visible.

## A.10 Division of labour

Two agents worked on this study: one performing experiments and owning technical truth, one organising results into the paper. Results were handed over through an append-only ledger, and the paper track verified each claim against the tests and sources rather than against the summaries. Three of the corrections above crossed that boundary in one direction or the other. Neither track was permitted to let the paper's needs drive a measurement.

The handoff has its own failure mode, and we found it. A figure derived from a design rule — not measured, not read off a source — crossed the boundary unlabelled, and was relayed onward as though it were established. It was wrong, and it would have understated the cost of a decision a person was about to take. The division of labour caught half of it: the rule that one track does not make recommendations on the other's behalf held, but a derived number travelling as background context does not trigger that check, because it does not look like a recommendation. Two symmetric habits followed: label a derivation as derived when handing it over, and ask who verified any figure before passing it to someone who will decide on it.

One distinction in that table is worth stating in prose, because it bears on what review can do. Three of the wrong assumptions were *written* — in a design document, where a reader could in principle have caught them. One was not: it existed only in the implementation, as a belief the code acted on and no document stated. **An assumption that exists only as code is unreviewable by construction**, and this one is also the single case that a green suite and a passing mutation battery both endorsed. Where the written errors were found by building, that one was found by printing a value.

A battery can also misreport its own coverage, and did. One surviving mutation was answered *behaviourally identical* and was in fact *unreachable*: it forged an identity at a point after an early return, so it changed nothing on any path. **A mutation that alters no behaviour tests nothing, and a battery that counts it as killed is overstating what it covers.** It was repositioned so that a real arm convicts it. This belongs beside the earlier case in which a green suite and a passing battery both endorsed a defect: instruments on this line have failed in both directions — missing an error, and appearing to have excluded one they never reached.

A second habit came out of the same slice and is cheap enough to state as a rule: *every arm asserts which capability it routed to*. Four arms in that set would have passed while exercising the wrong path had they not.

A note on the price of the mechanism, offered as an observation rather than a finding: its estimate has been corrected four times, each time upward, the last when implementing coverage moved the worst case from one observation per binding to one per addressable resource. That is one quantity, four data points, all from one track's estimates. It supports *this price was repeatedly underestimated here*; it does not support any claim about estimation in general.

One further observation about habits of this kind, and it is the only one here for which we have direct evidence rather than a tally. A rule adopted after one correction did not prevent the next instance of the same error: the rule *ask whether a sentence claims something was judged or that it is impossible* was written down one exchange before a sentence claiming there was no third option in the accounting. **Rules fail at the moment of writing because the objective function of writing runs against them** — a sentence composed to close off an objection is rewarded for

sounding closed. What corrected it was not the rule but an action taken by someone else. We report this narrowly: we can show that a remembered rule failed once, here, with the record to hand. We cannot show which corrective actions work in general, and an earlier draft of this appendix claimed we could.

## B Reproducibility

**A test outcome here is not a pure function of the commit**, and the limit is worth stating where a reader would otherwise assume it away: one test walks the real TypeScript import graph from the daemon’s entry point, so it cannot survive a checkout whose dependencies are symlinked, and a detached worktree of this repository diverges from the main tree by exactly that file’s assertions. **So green at commit *X* is reproducible only with the local dependency state described, not from *X* alone**, and a count of failing assertions — which this work uses as evidence in one place — needs its method stated beside it. Everything else below is checkable from the repository.

Every result is a test in the repository. Experiments are named v58–v73, probe-3–probe-6, and eight later slices — u6, vb, vb2, pp1, ov1, e2e1, e2e1-k8 and w2p — and each arm names its commit in the results ledger. Fixtures are in `tests/`, batteries in `tests/batteries/`, and all two hundred and sixty-nine measured arms are lines in `docs/research/results.jsonl`, from which Figure 4 and Table 1 are generated rather than drawn. That column is checked rather than trusted, by a test in the repository and again by the generator that prints it: the recorded commit must have *touched* one of the experiment’s test files. Three experiments had recorded their own slice’s parent — a value indistinguishable from a valid one by inspection — and an earlier form of the check, asking only whether the commit *contained* such a file, could not have caught them all (Appendix A). The table generator withholds an anchor it cannot verify rather than printing one a reader cannot check out. The figure generator prints no anchors and applies a different rule: it drops any experiment whose test exists only in a working tree, since such a measurement is real but not reproducible from any commit. The ledger’s own convention is that a missing key means *not measured* and never *measured false*.

Table 1 lists every experiment with its commit and the shape of its arms, and is rendered from that ledger rather than maintained by hand; regenerating it after a new experiment is a single command.

**Table 1.** Every experiment, its commit, and the shape of its arms. Generated from the results ledger. *cf* counts counterfactual arms — a real component evaluated under a mode the runtime does not produce.

| experiment          | commit  | arms | ctrl | cf | boundary           |
|---------------------|---------|------|------|----|--------------------|
| v58 <sup>†</sup>    | ad08e95 | 3    | 1    | –  | run                |
| probe-3             | aba6138 | 2    | 1    | –  | run                |
| probe-4             | d5b5b66 | 5    | 2    | –  | run                |
| probe-5             | f642b20 | 5    | 2    | 3  | run                |
| probe-6             | 98a4e25 | 5    | 2    | 2  | run                |
| v59-1               | 3f6cad2 | 5    | 4    | –  | bound              |
| v59-2               | bbecc27 | 1    | 0    | –  | bound              |
| v60                 | 5bff230 | 4    | 3    | –  | bound              |
| v61                 | 5bff230 | 5    | 1    | –  | resume             |
| v62                 | 582d3ed | 4    | 3    | –  | run, resume        |
| v63                 | 7f6717f | 5    | 2    | –  | run                |
| v64 <sup>†</sup>    | 4a8694f | 3    | 1    | –  | run                |
| v65 <sup>†</sup>    | 110ea62 | 12   | 3    | –  | run, resume        |
| v66 <sup>†‡</sup>   | 125b392 | 9    | 2    | 1  | run                |
| v67 <sup>‡</sup>    | 10fed3c | 9    | 3    | –  | run                |
| v68 <sup>†‡</sup>   | f3e6e4a | 7    | 3    | –  | run                |
| v69 <sup>†</sup>    | d0c1f86 | 7    | 3    | –  | run                |
| v70                 | 0ac7c8e | 4    | 1    | –  | run                |
| v71                 | 8f8e3e8 | 3    | 1    | –  | resume             |
| v72-1 <sup>‡</sup>  | 22c74d8 | 4    | 2    | –  | run                |
| v72-2               | 151ac7e | 4    | 1    | –  | run                |
| v72-3 <sup>†‡</sup> | d00e369 | 3    | 1    | –  | run                |
| v72-4 <sup>†</sup>  | 45812d1 | 3    | 1    | –  | run                |
| v73 <sup>†</sup>    | 7037bc2 | 31   | 10   | 19 | run, bound, resume |
| u6 <sup>†</sup>     | 9a8b194 | 12   | 2    | –  | run, resume        |
| vb                  | f812448 | 27   | 9    | –  | n/a                |
| vb2                 | b142101 | 40   | 10   | –  | n/a                |
| pp1                 | 626be04 | 7    | 3    | –  | resume             |
| ov1 <sup>†</sup>    | 198c69c | 20   | 10   | 20 | run, resume        |
| e2e1-k8             | e5cf8c2 | 3    | 1    | –  | resume             |
| e2e1 <sup>†</sup>   | 994ecc7 | 14   | 3    | –  | run, resume        |
| w2p                 | 343b418 | 3    | 1    | –  | resume             |
| k8r                 | 9297c6c | 3    | 1    | –  | resume             |
| total               |         | 272  | 93   | 45 |                    |

<sup>†</sup>carries an arm whose prediction did not hold. <sup>‡</sup>carries an arm nobody predicted. The two are different failures and are counted separately (Appendix A); refuted predictions are retained in place.

**Table 2.** What the approval mechanism can see, per write capability outside the filesystem. Read from each provider’s own capability description.

| capability                  | what the mechanism sees                  | is that the effect? |
|-----------------------------|------------------------------------------|---------------------|
| repo.stage                  | paths                                    | yes                 |
| repo.commit.create          | <i>nothing</i>                           | —                   |
| repo.branch.create          | the ref                                  | yes                 |
| repo.checkout               | the ref                                  | yes                 |
| repo.push                   | the ref                                  | yes                 |
| shell.run,<br>process.start | run.build,<br>the working directory only | no                  |